

Linked Lists

COMP 2402

Abstract Data Types & Algorithms

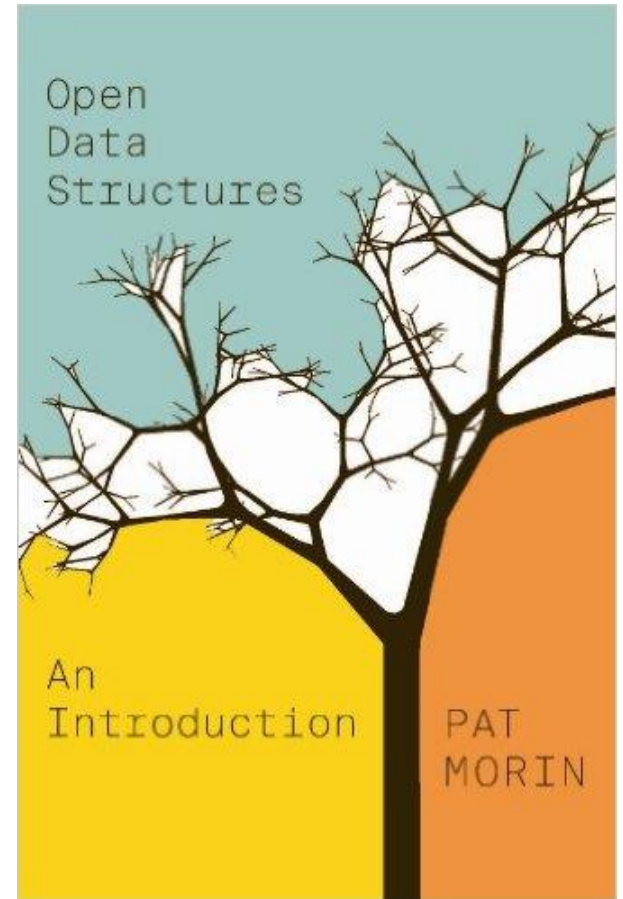
Readings

This class

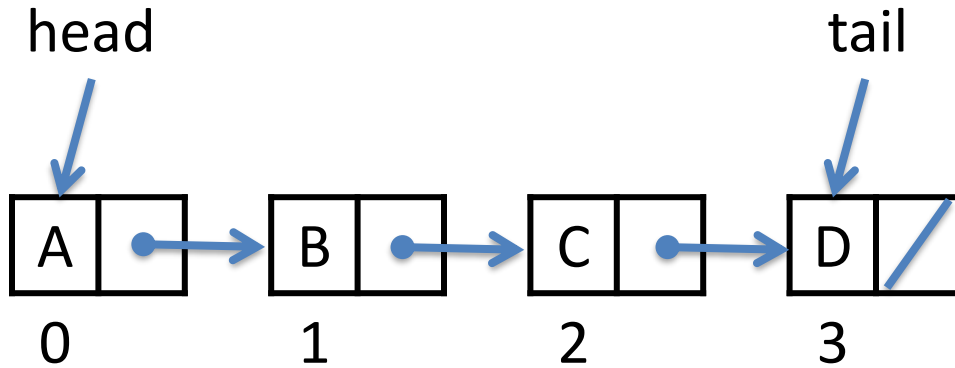
- ods 3 Linked Lists

Next Class

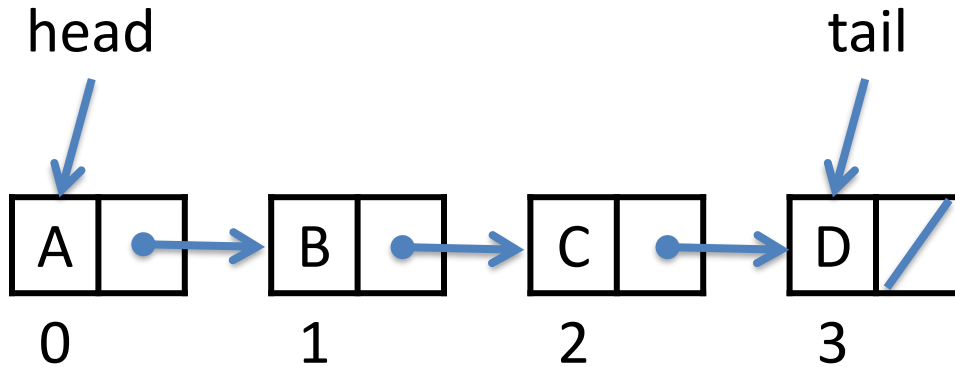
- ods 4 Skiplists



Singly-Linked Lists



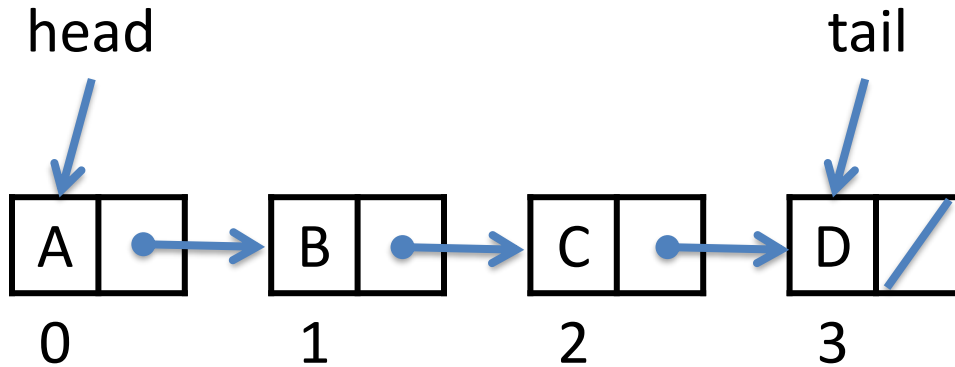
Singly-Linked Lists



A singly-linked list (**SSList**) is a sequence of Nodes

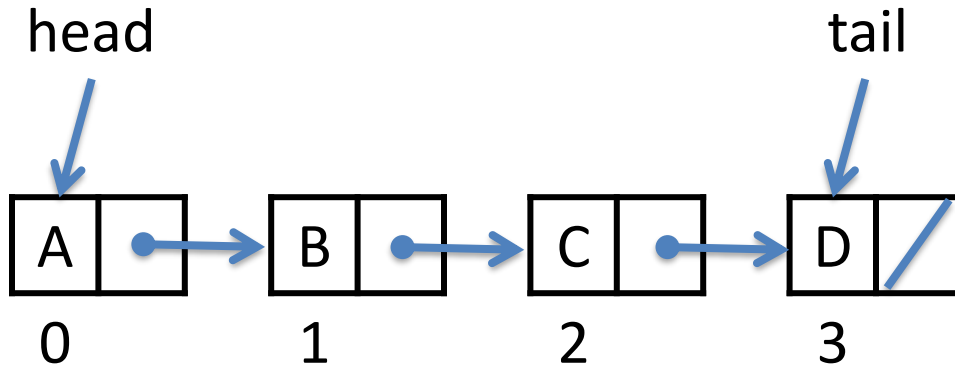
A **Node** stores **data** and a **reference to the next node** (or null if there is no next node)

Singly-Linked Lists

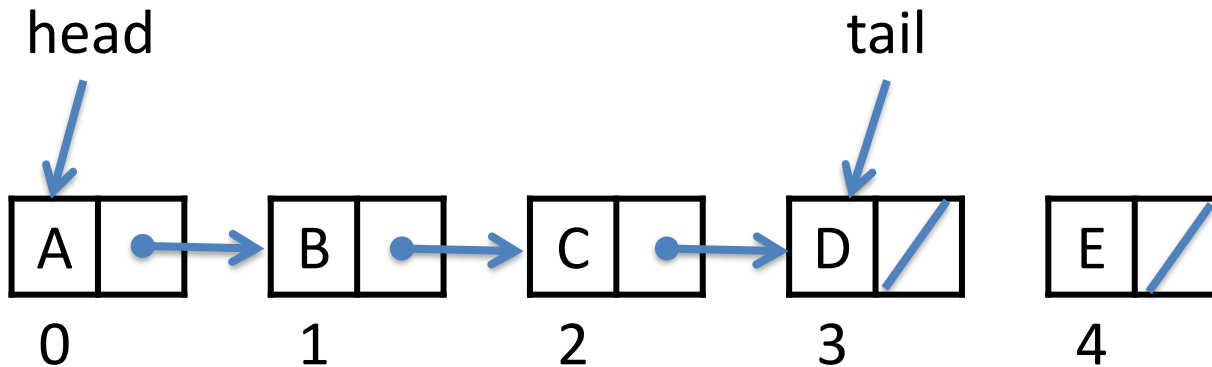


Add an element to the end of the list

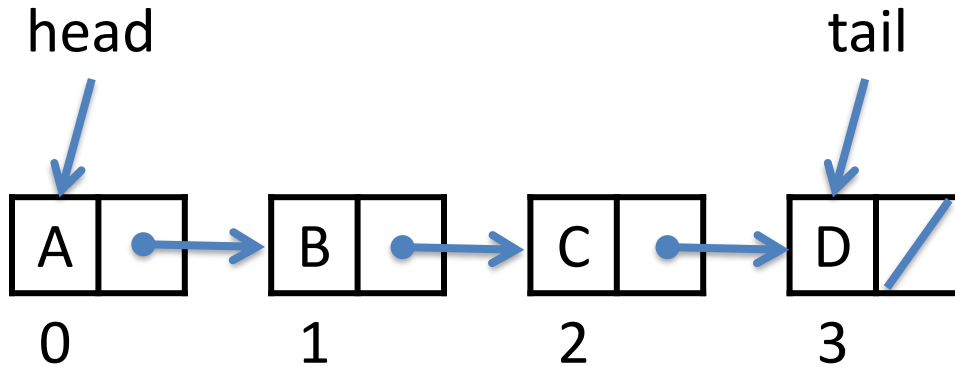
Singly-Linked Lists



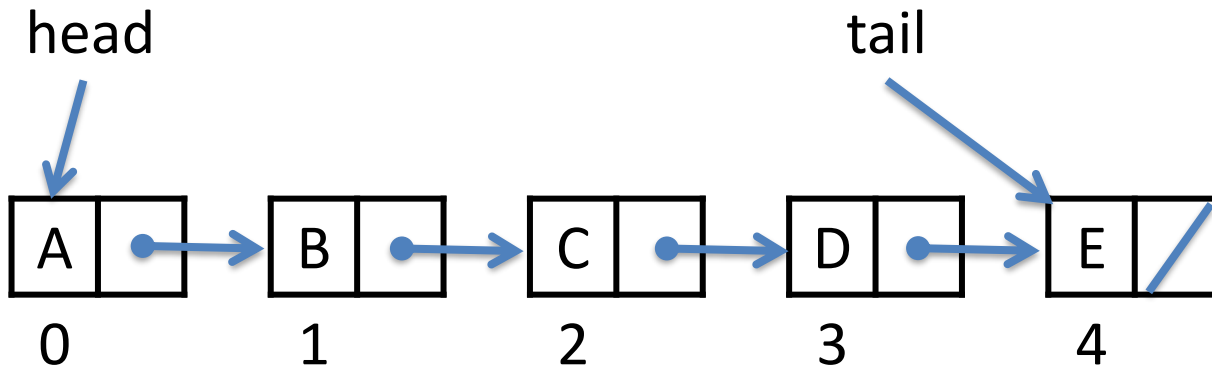
Add an element to the end of the list



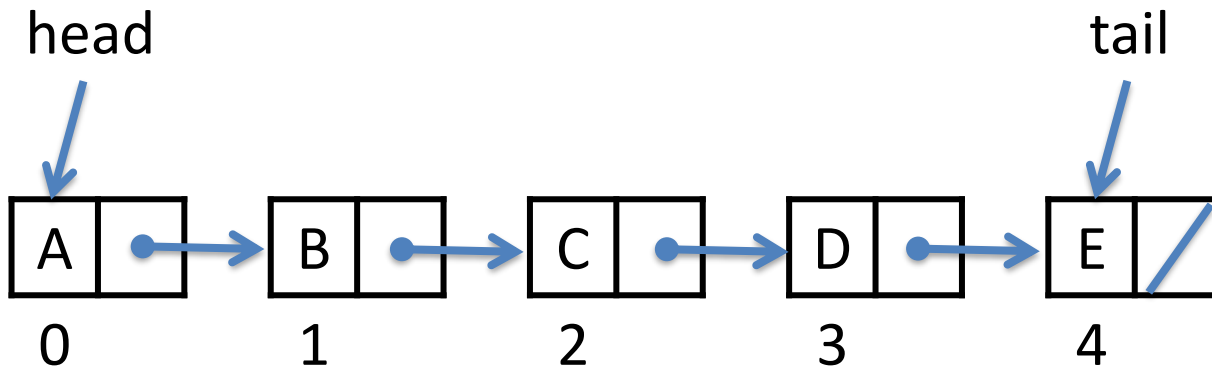
Singly-Linked Lists



Add an element to the end of the list

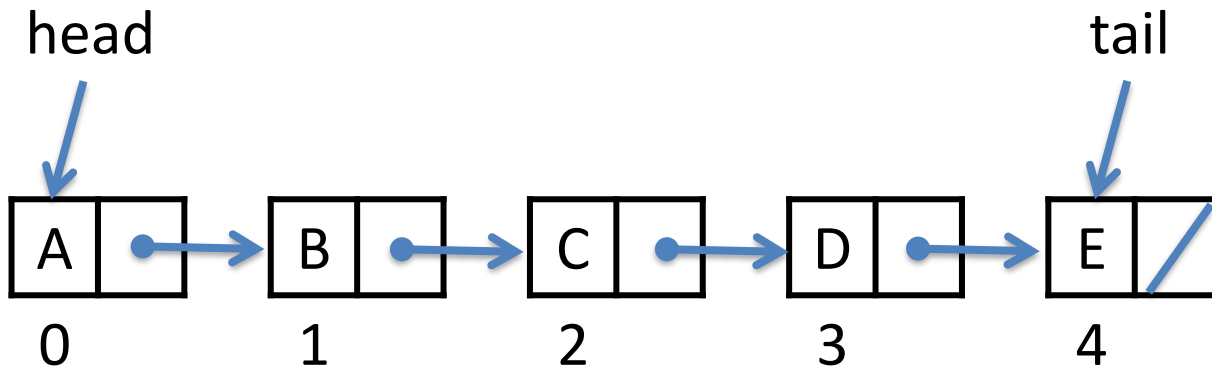


Singly-Linked Lists

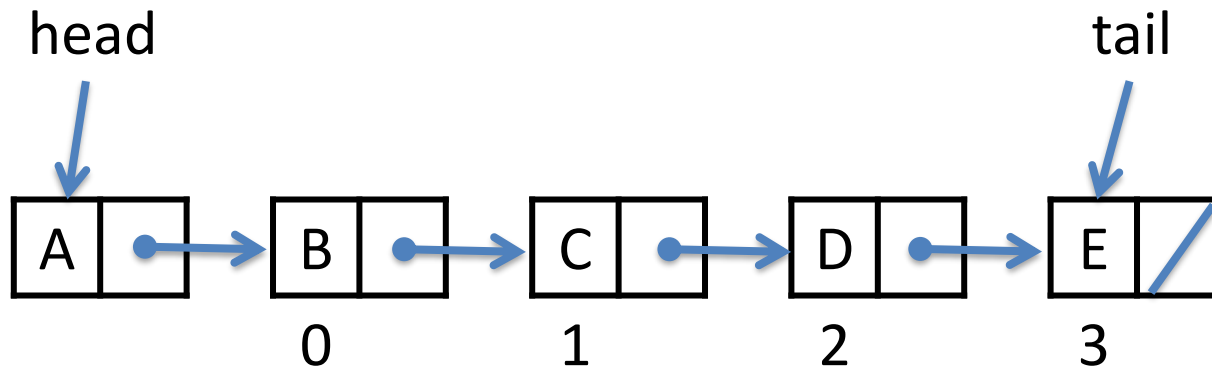


Remove the first element in the list

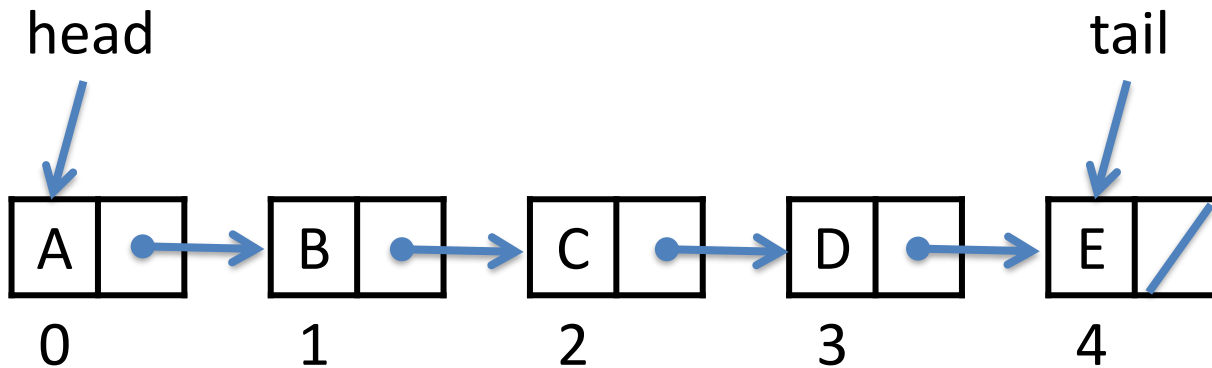
Singly-Linked Lists



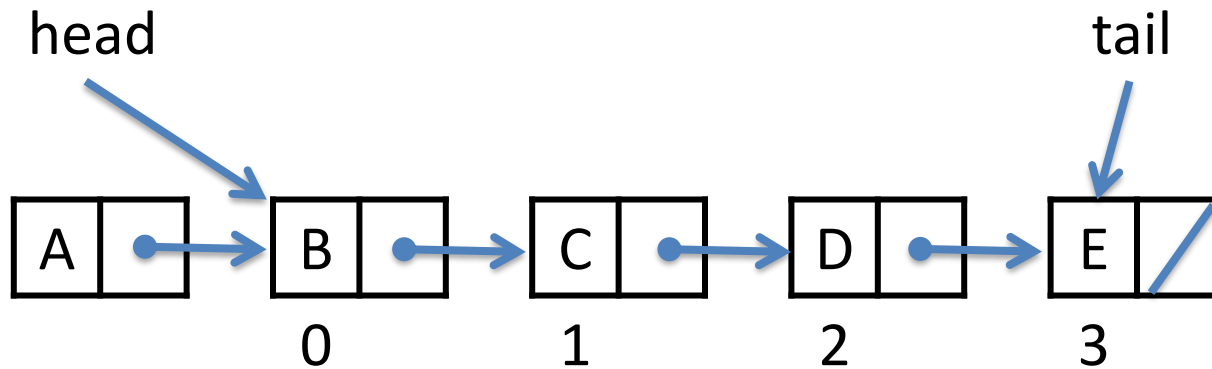
Remove the first element in the list



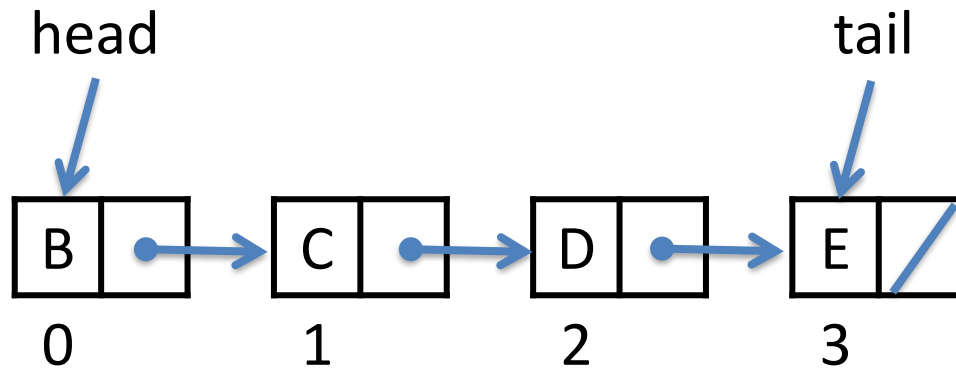
Singly-Linked Lists



Remove the first element in the list

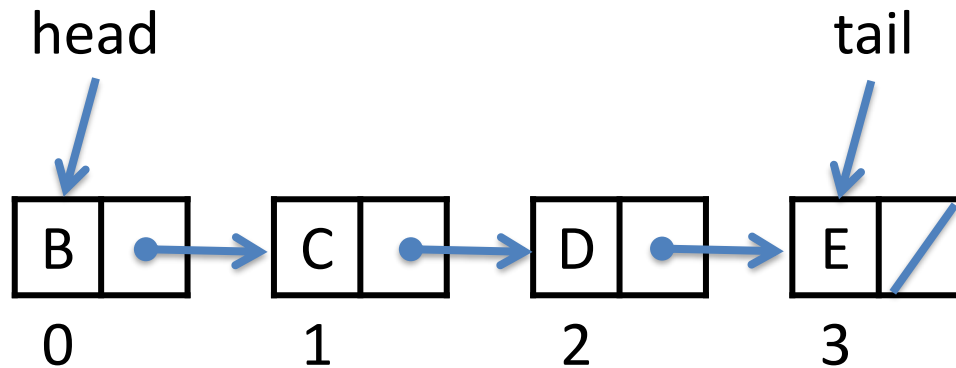


Singly-Linked Lists

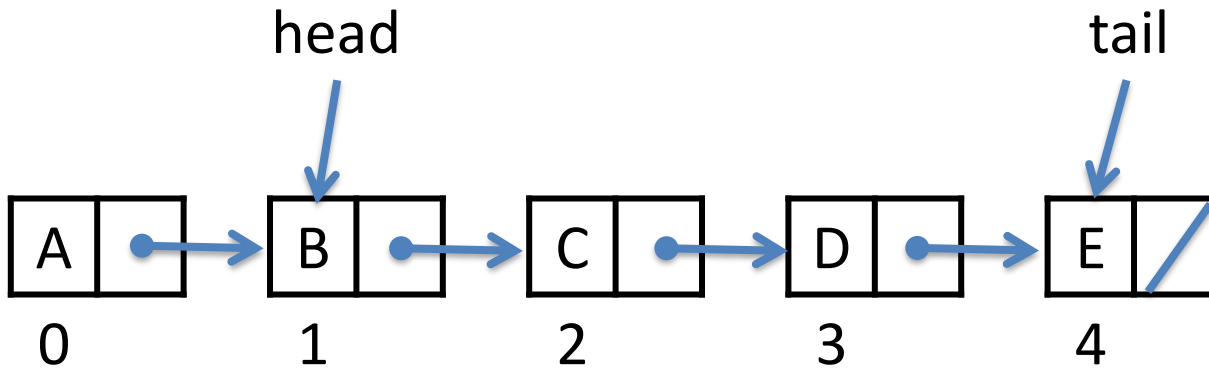


Add an element to the front of the list

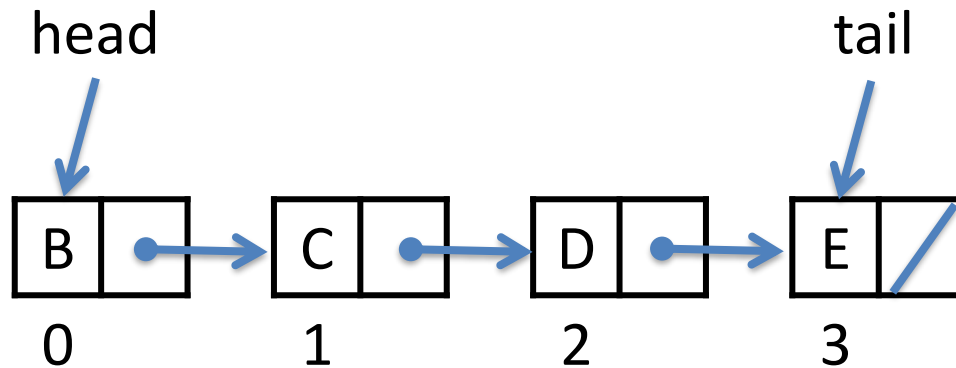
Singly-Linked Lists



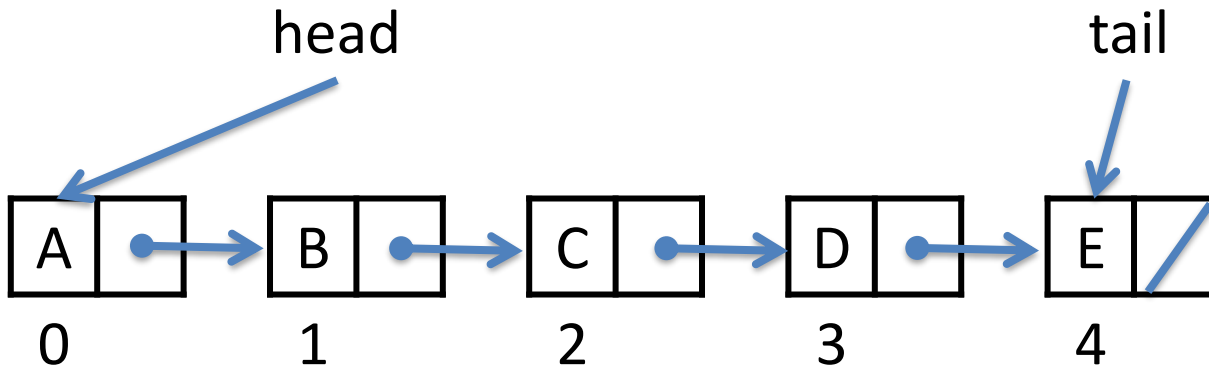
Add an element to the front of the list



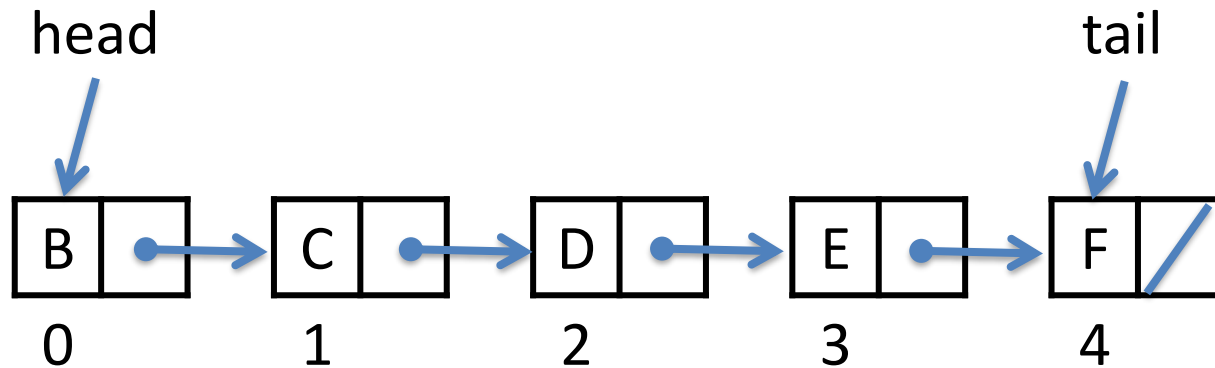
Singly-Linked Lists



Add an element to the front of the list

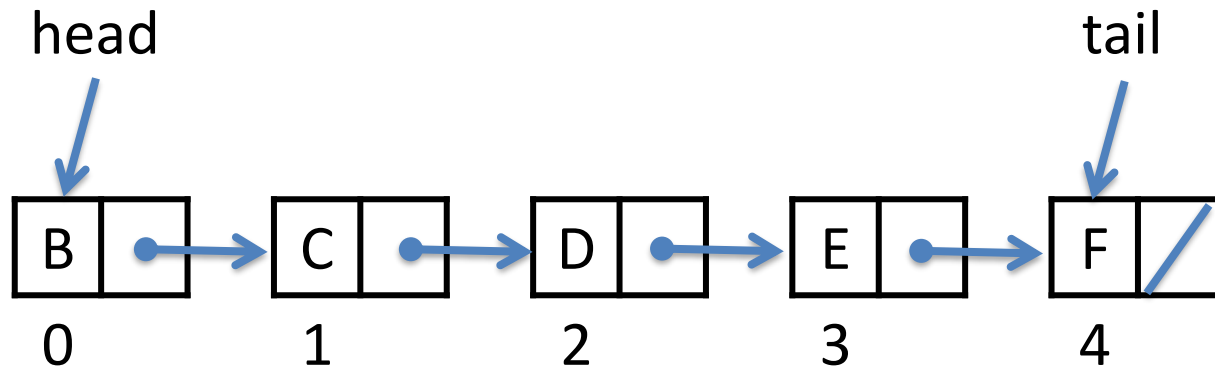


Singly-Linked Lists



How do we remove from the end of the list?

Singly-Linked Lists

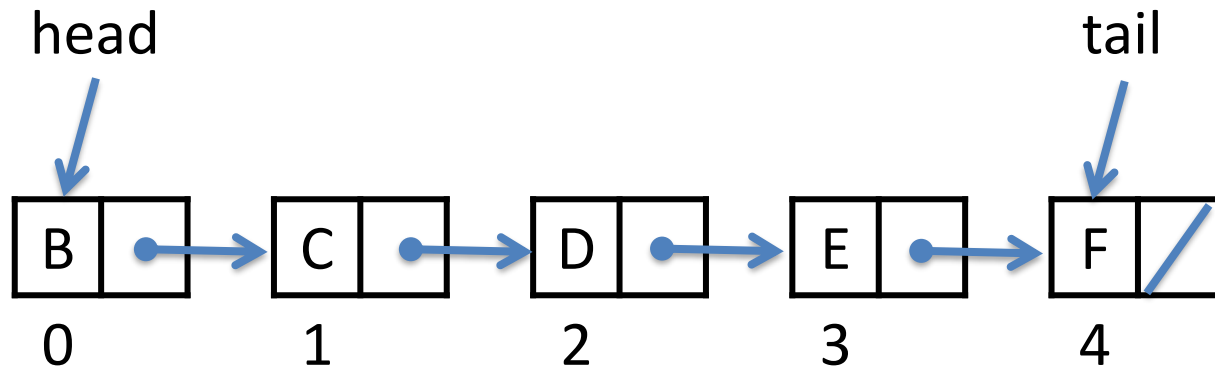


How do we remove from the end of the list?

We have to walk from the front (head) to the second last node in order to set its **next** field to **null**

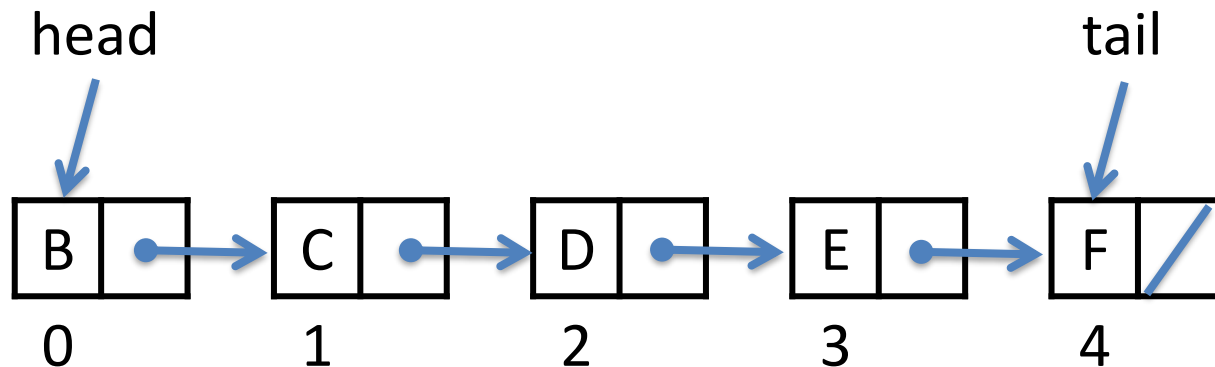
Cost is $O(n)$

Singly-Linked Lists



How do we add/remove from everywhere else in the list?

Singly-Linked Lists



How do we add/remove from everywhere else in the list?

We have to walk from the front (head) of the list to where we want to add/remove a node

Cost is $O(1 + i)$ for index i (worst case $O(n)$)

Singly-Linked List

Theorem 3.1: An SLList implements the Stack and FIFO Queue interface. Adding and removing to these data structures costs $O(1)$ time per operation.

There is no costly "grow" or "shrink" operations like we had with arrays.

Stack – push and pop

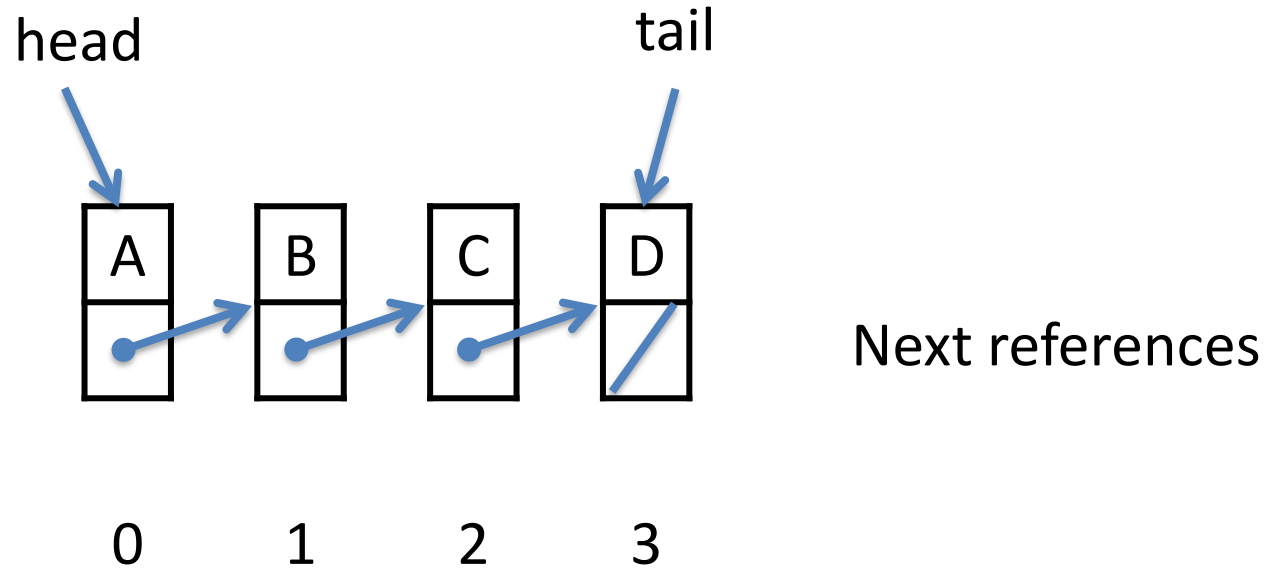
FIFO Queue – enqueue and dequeue

Singly-Linked List

Theorem 3.1: An SLList implements the Stack and FIFO Queue interface. Adding and removing to these data structures costs $O(1)$ time per operation.

Using a singly-linked list for a List implementation will be inefficient. The $\text{get}(i)/\text{set}(i, x)$ operations will also be $O(1 + i)$ and so $O(n)$ in general.

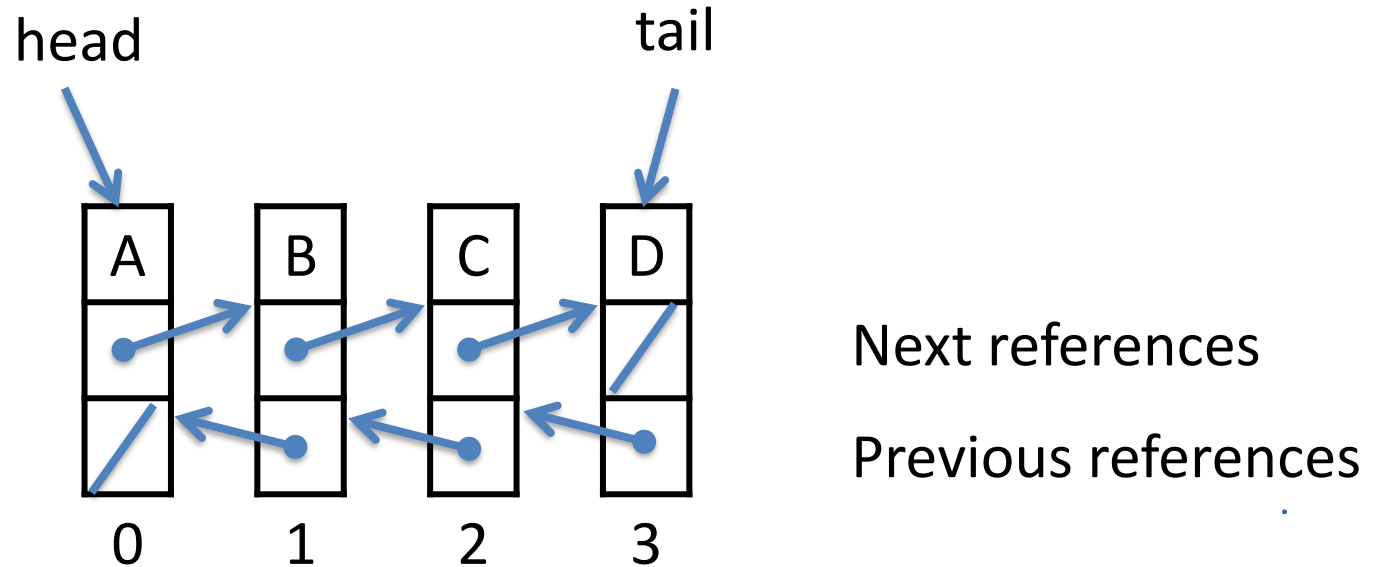
Doubly-Linked Lists



Each node has

- Data
- A reference to the next Node

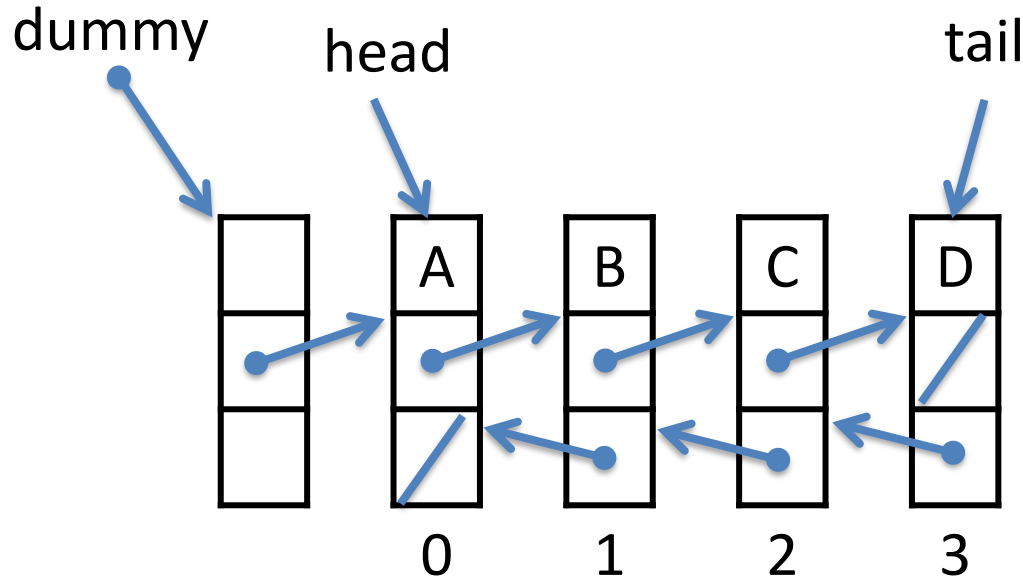
Doubly-Linked Lists



Each node has

- Data
- A reference to the next Node
- A reference to the previous Node

Doubly-Linked Lists



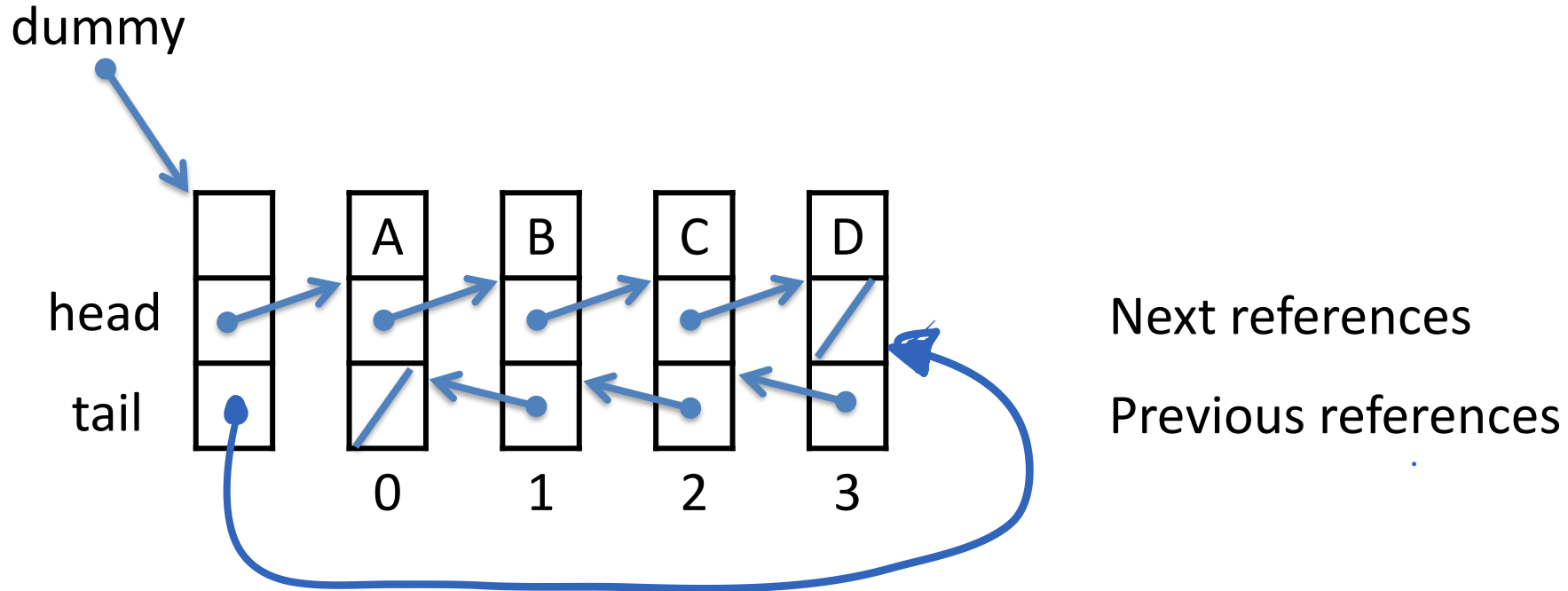
Next references

Previous references

Each node has

- Data
- A reference to the next Node
- A reference to the previous Node

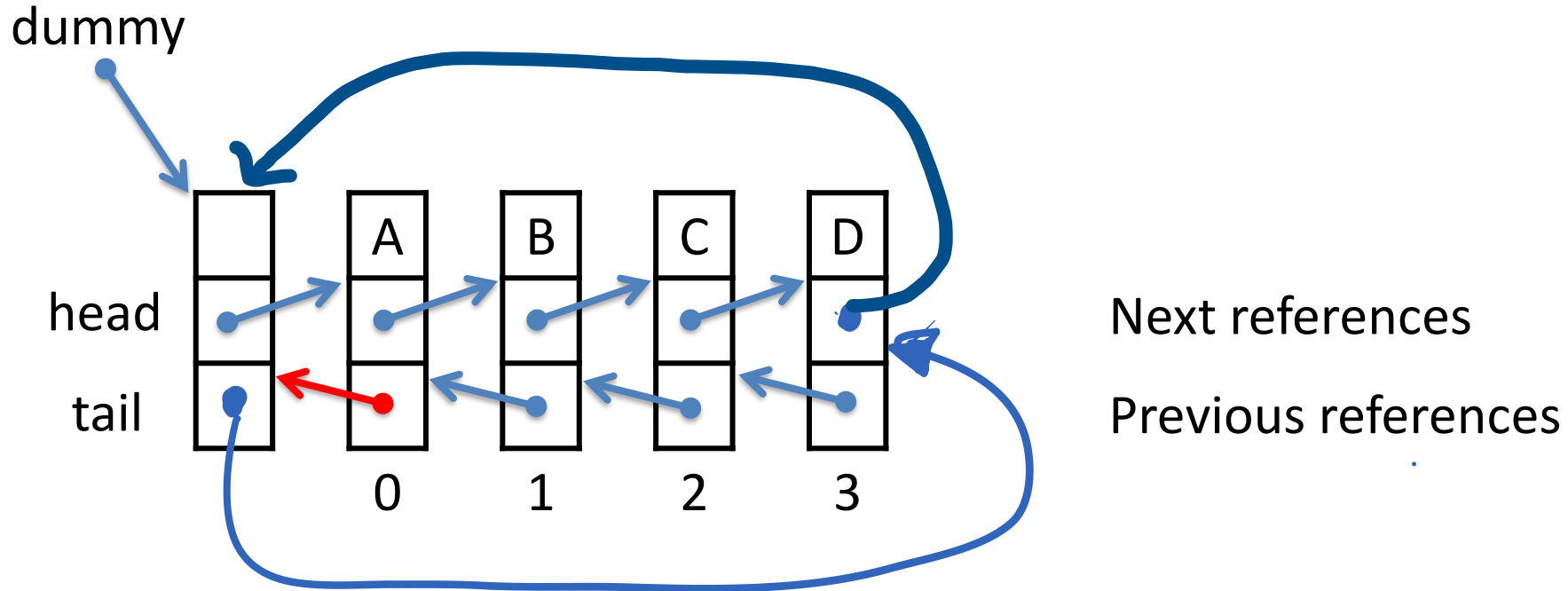
Doubly-Linked Lists



Each node has

- Data
- A reference to the next Node
- A reference to the previous Node

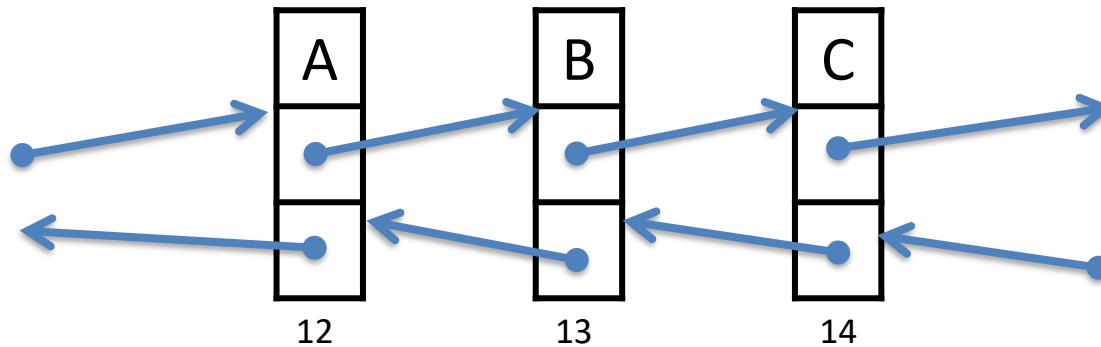
Doubly-Linked Lists



Each node has

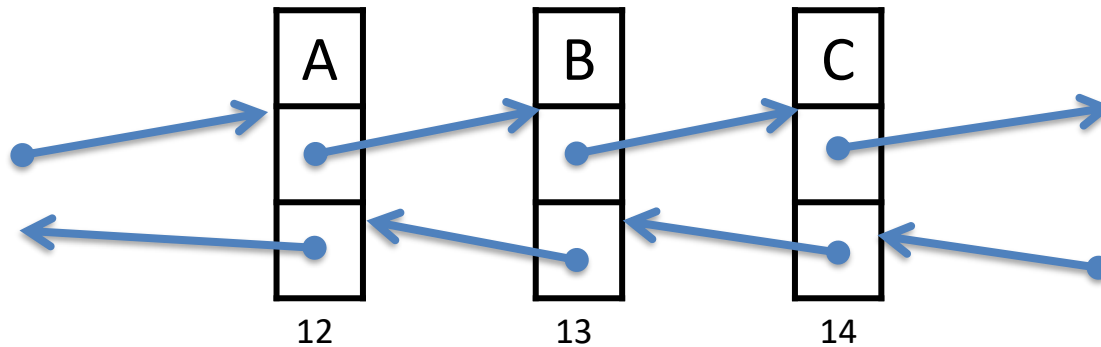
- Data
- A reference to the next Node
- A reference to the previous Node

Doubly-Linked Lists



Finding a node in the list

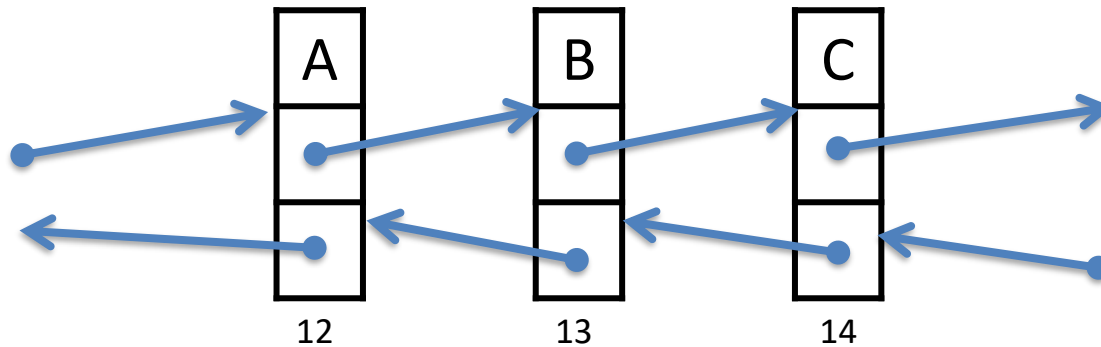
Doubly-Linked Lists



Finding a node in the list

- `getNode(i)` – returns the node at list index i

Doubly-Linked Lists

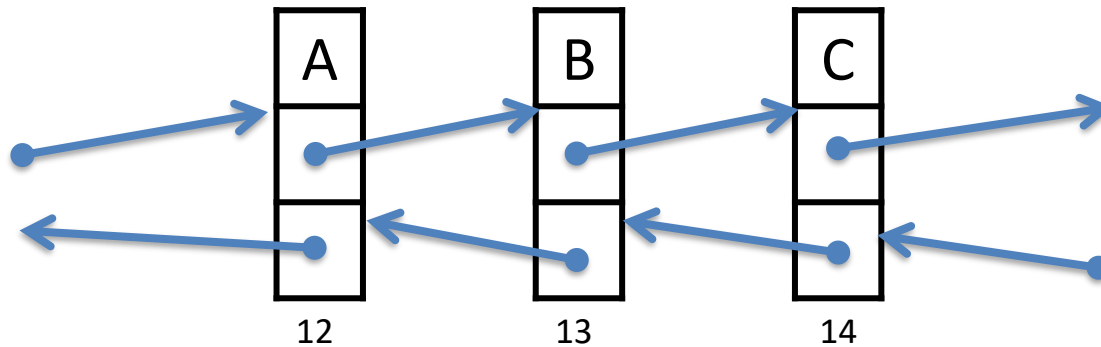


Finding a node in the list

- `getNode(i)` – returns the node at list index i

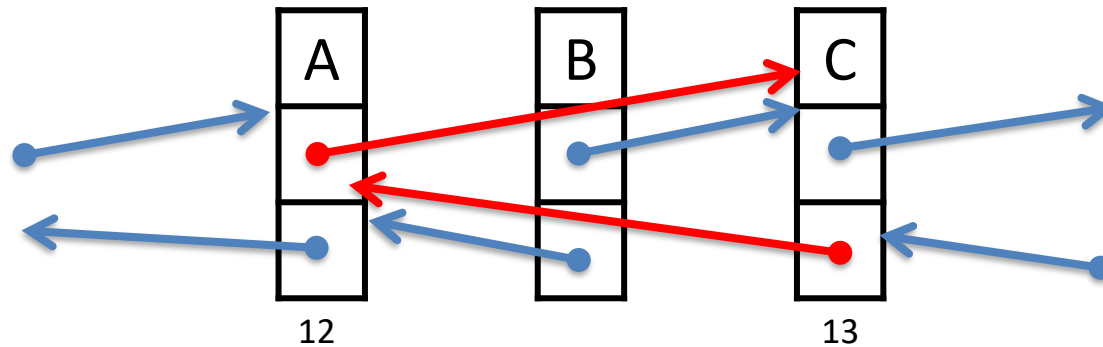
has $O(1 + \min\{i, n - i\})$ runtime

Doubly-Linked Lists



Removing node 13

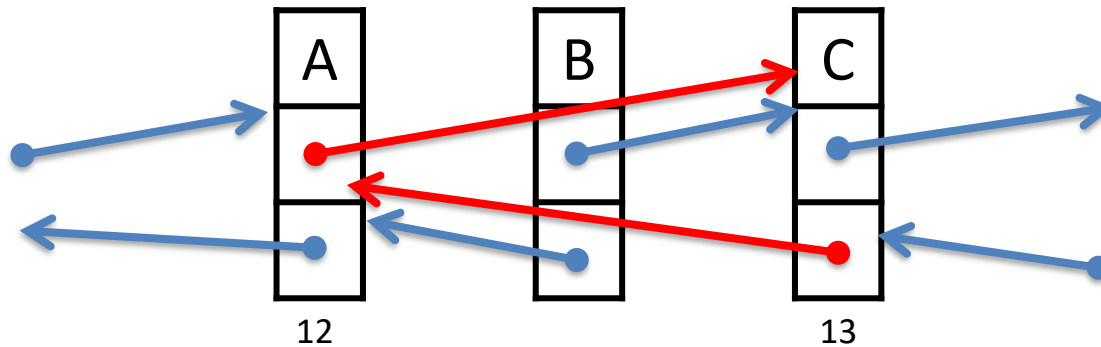
Doubly-Linked Lists



Removing node 13

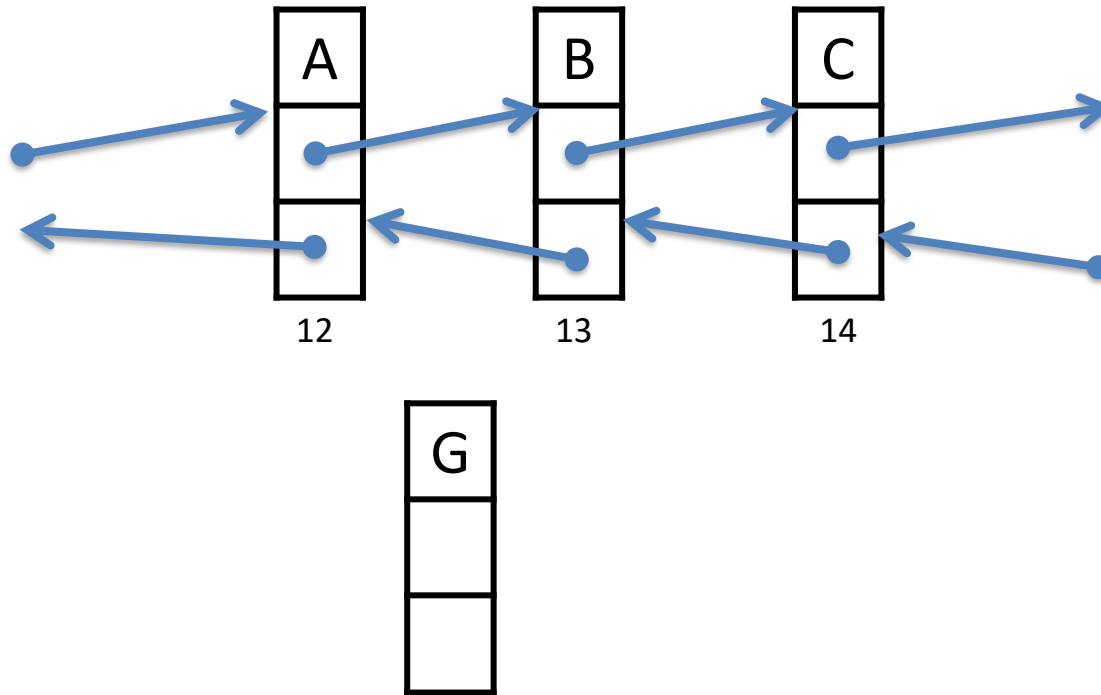
- `remove( 13)` calls `getNode(13)` and then modifies TWO references
 - One in previous node and one in next node
- Update size of list `n`

Doubly-Linked Lists



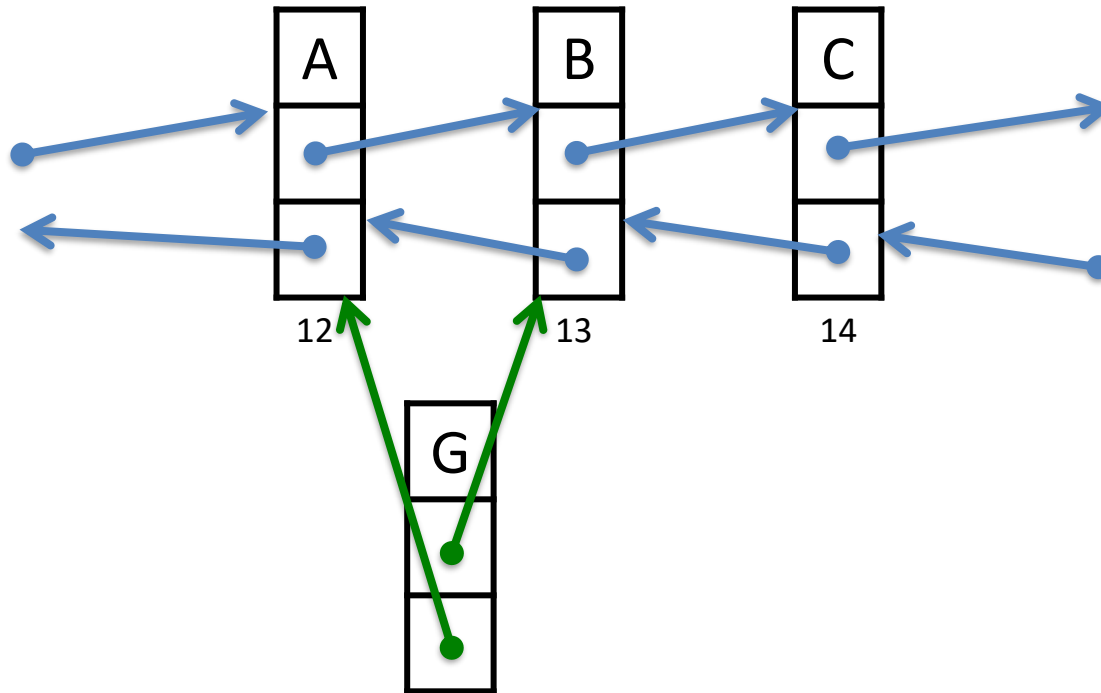
What is the cost of removing from the list?

Doubly-Linked Lists



Adding a node before node 13

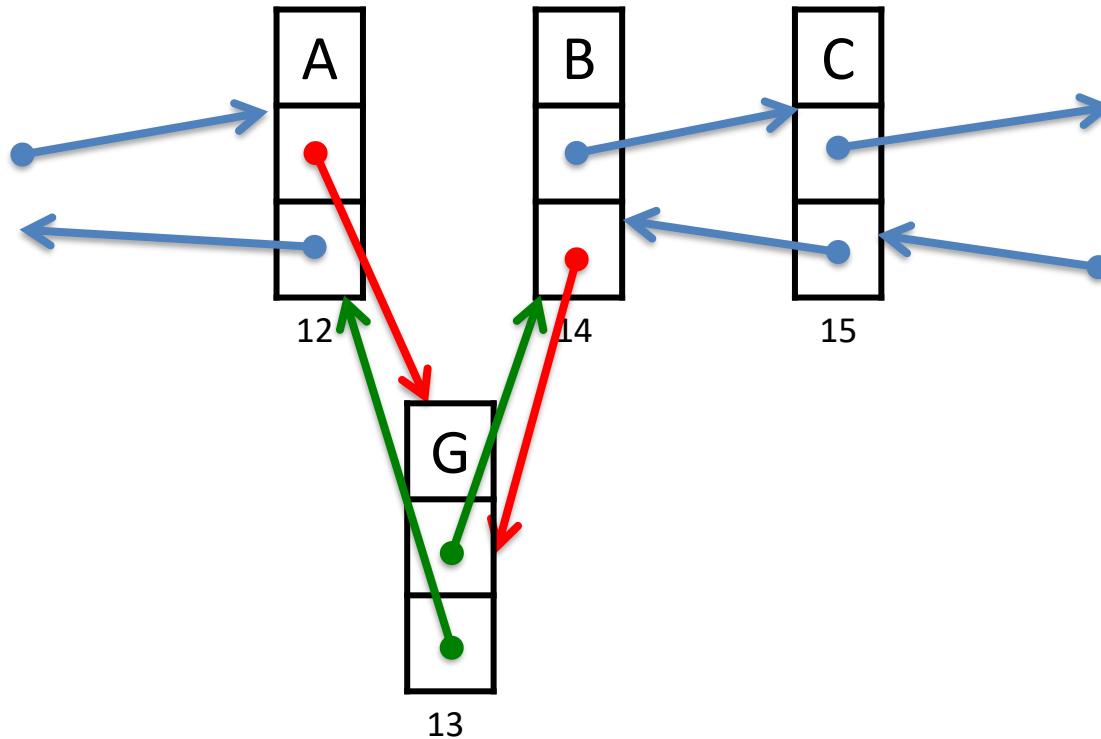
Doubly-Linked Lists



Adding a node before node 13

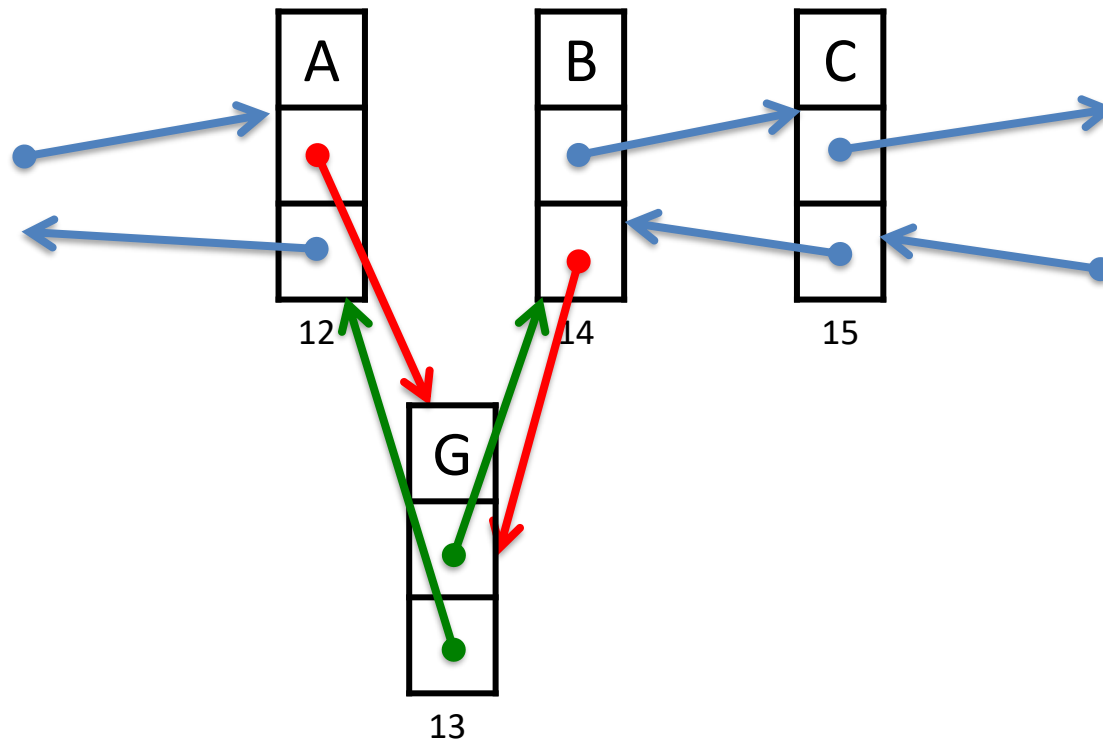
- Find node 13 and set new nodes references

Doubly-Linked Lists



What is the cost of adding to the list?

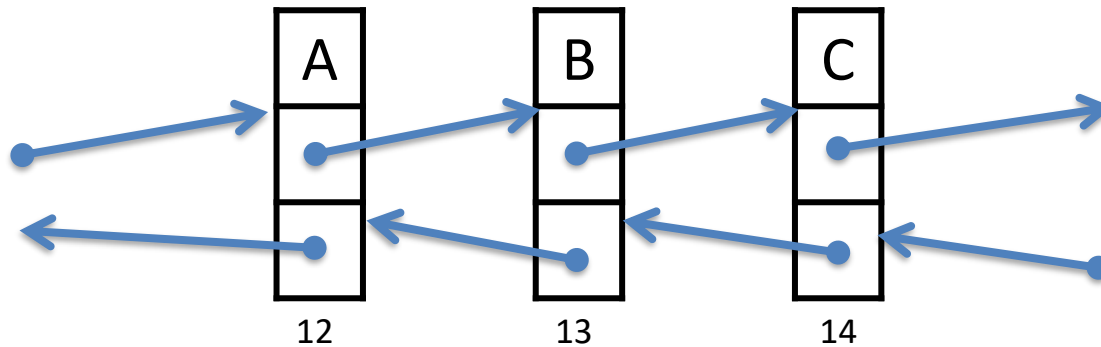
Doubly-Linked Lists



Adding a node before node 13

- Find node 13 and set new nodes references
- Update node 13's previous and node 12's next
- Update n

Doubly-Linked Lists



Cost of add/remove is dominated by the cost to
findNode()

$$O(1 + \min\{i, n - i\})$$

Doubly-Linked List

Theorem 3.2: A DLList implements the List interface. The `get(i)`, `set(i,x)`, `add(i,x)` and `remove(i)` operations all run in

$$O(1 + \min\{i, n - i\})$$

time per operation.

Doubly-Linked List

How much space does the DLList use?

Space-Efficient Linked List

Consider a doubly-linked list where we store a **block of data** instead of just one element

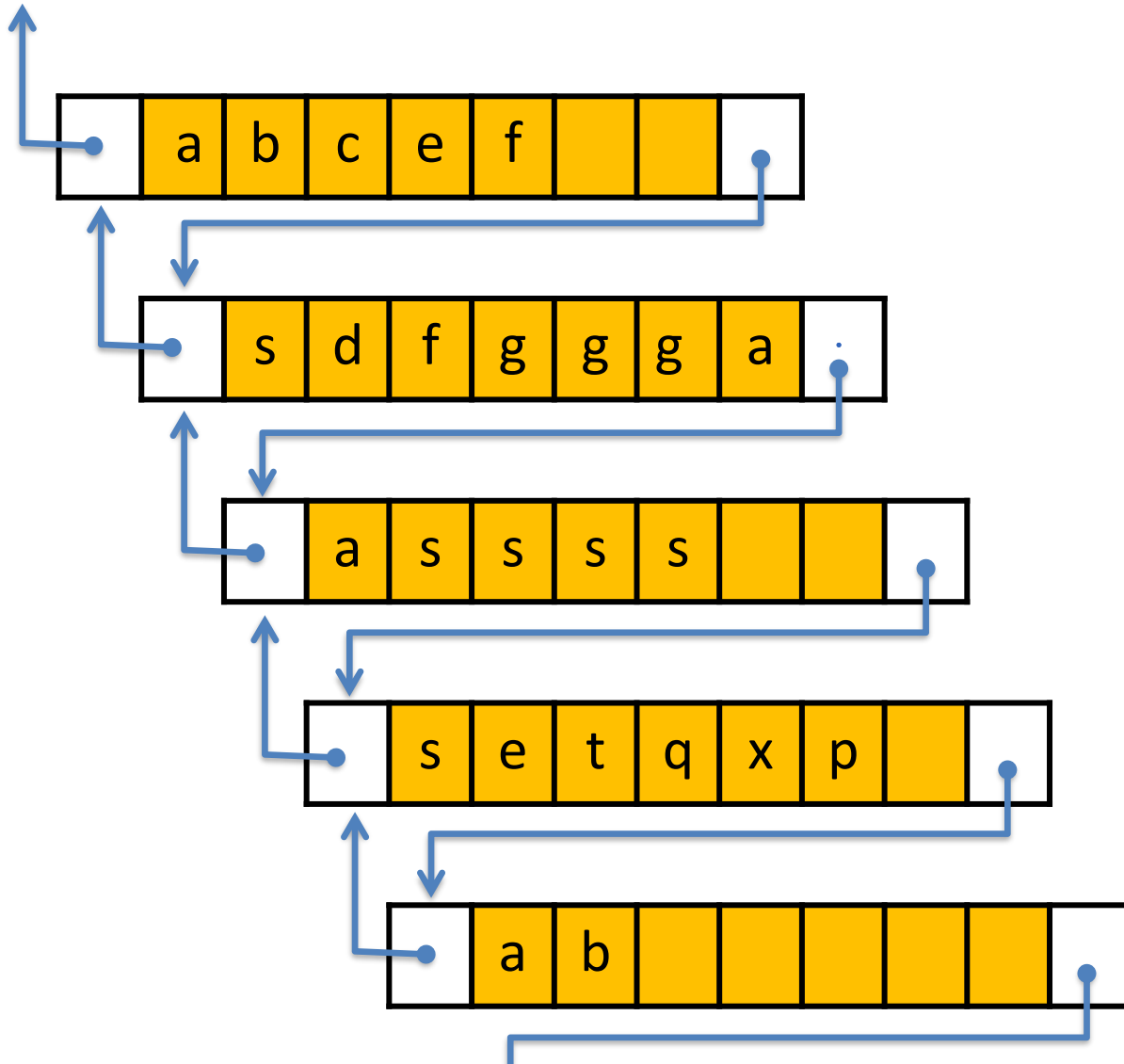
For fixed **block size** b , each block will store *up to* $b + 1$ elements using a deque

A **BDeque** is a bounded deque

- Has fixed length (never grows or shrinks)
- Constant runtime for all Deque operations (add/remove from the front/back)

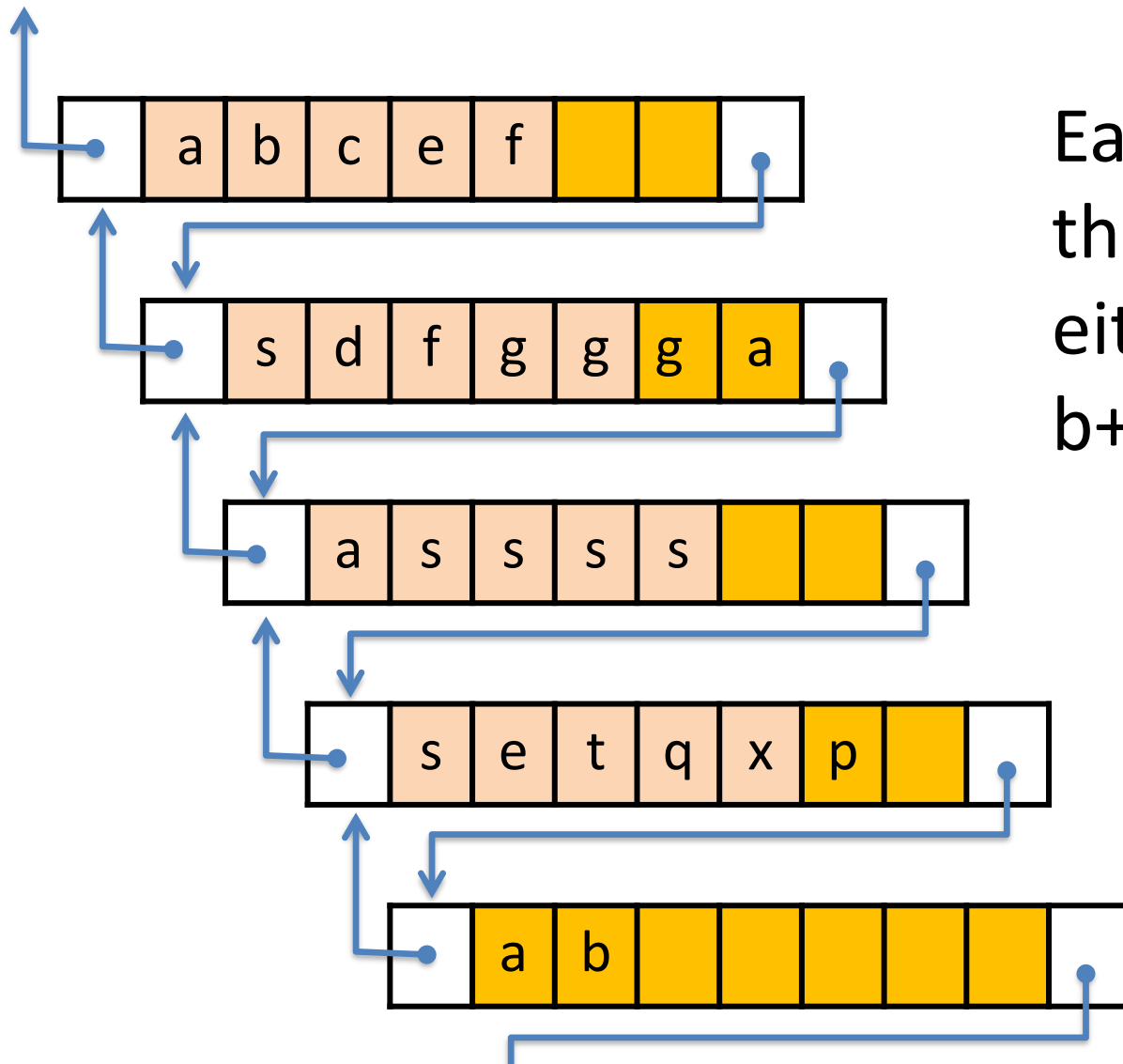
Space-Efficient Linked List

An **SEList** is a doubly-linked list of blocks



Space-Efficient Linked List

An **SEList** is a doubly-linked list of blocks

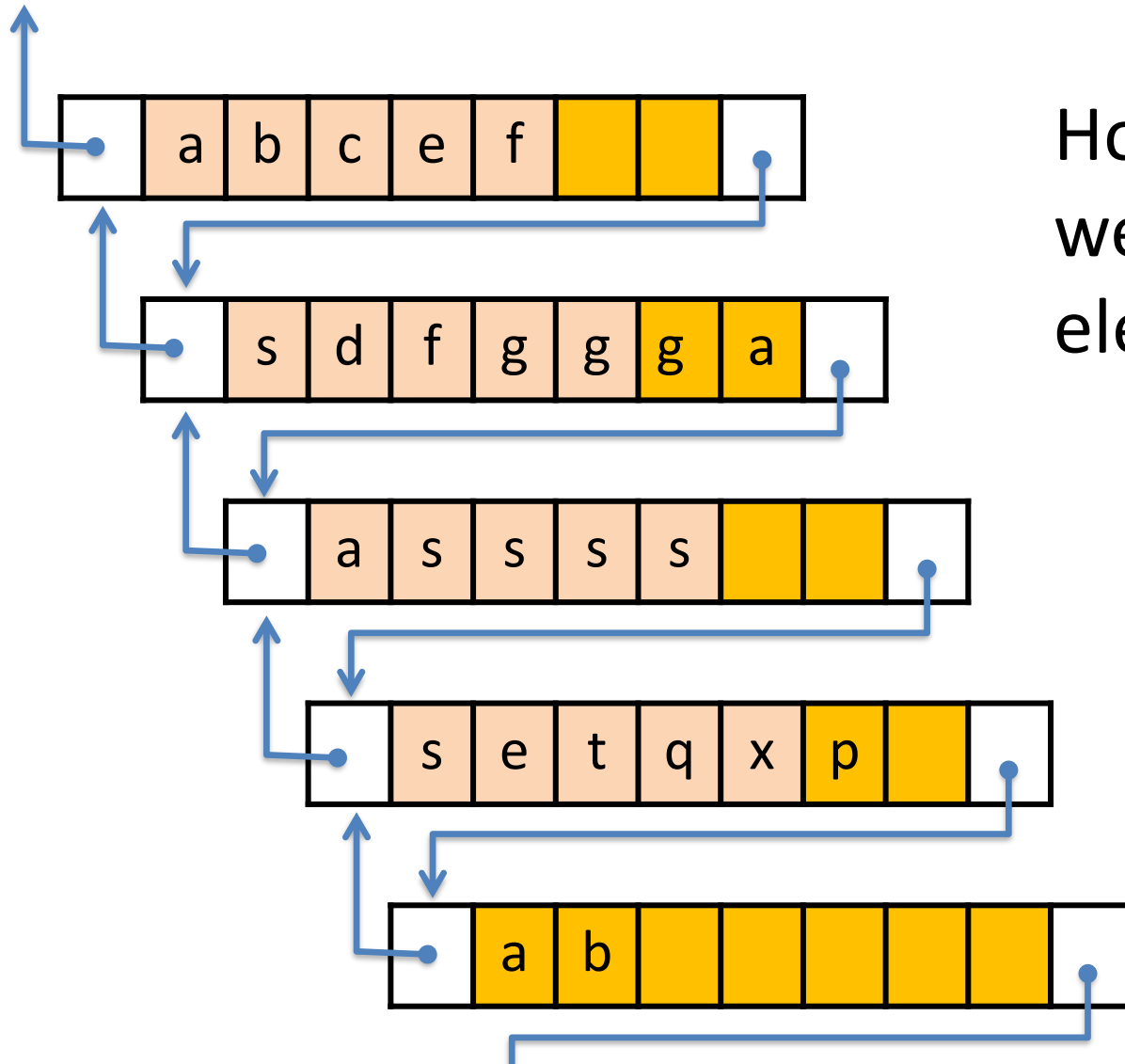


Each block except the last must have either $b-1$, b or $b+1$ elements

← last block

Space-Efficient Linked List

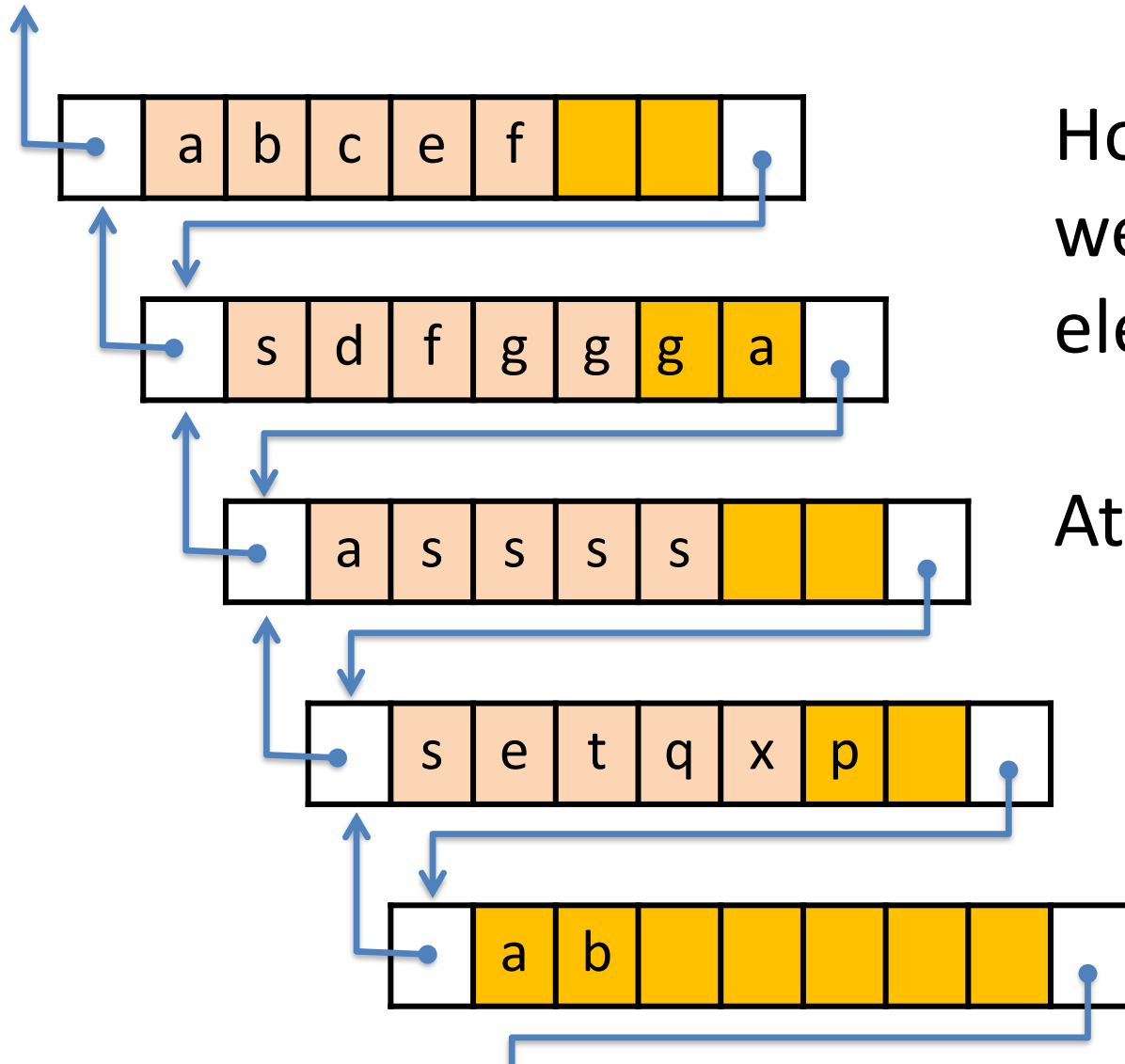
An **SEList** is a doubly-linked list of blocks



How many blocks do we need to store n elements?

Space-Efficient Linked List

An **SEList** is a doubly-linked list of blocks



How many blocks do we need to store n elements?

At most

$$\frac{n}{b-1} + 1 \in O(n/b)$$

Space-Efficient Linked List

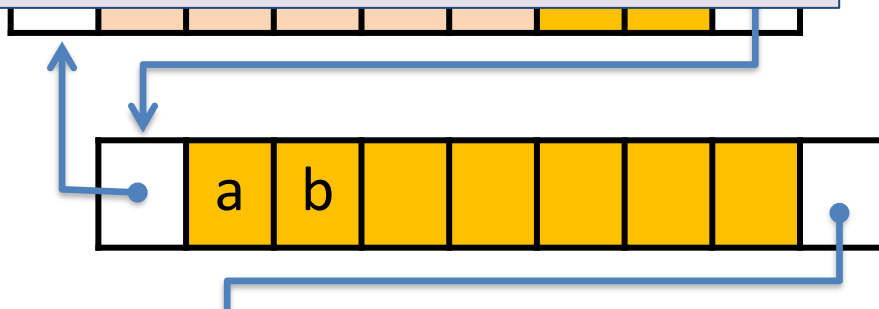
What is the space requirements to store n elements in the list?

blocks

How many blocks do we need to store n elements?

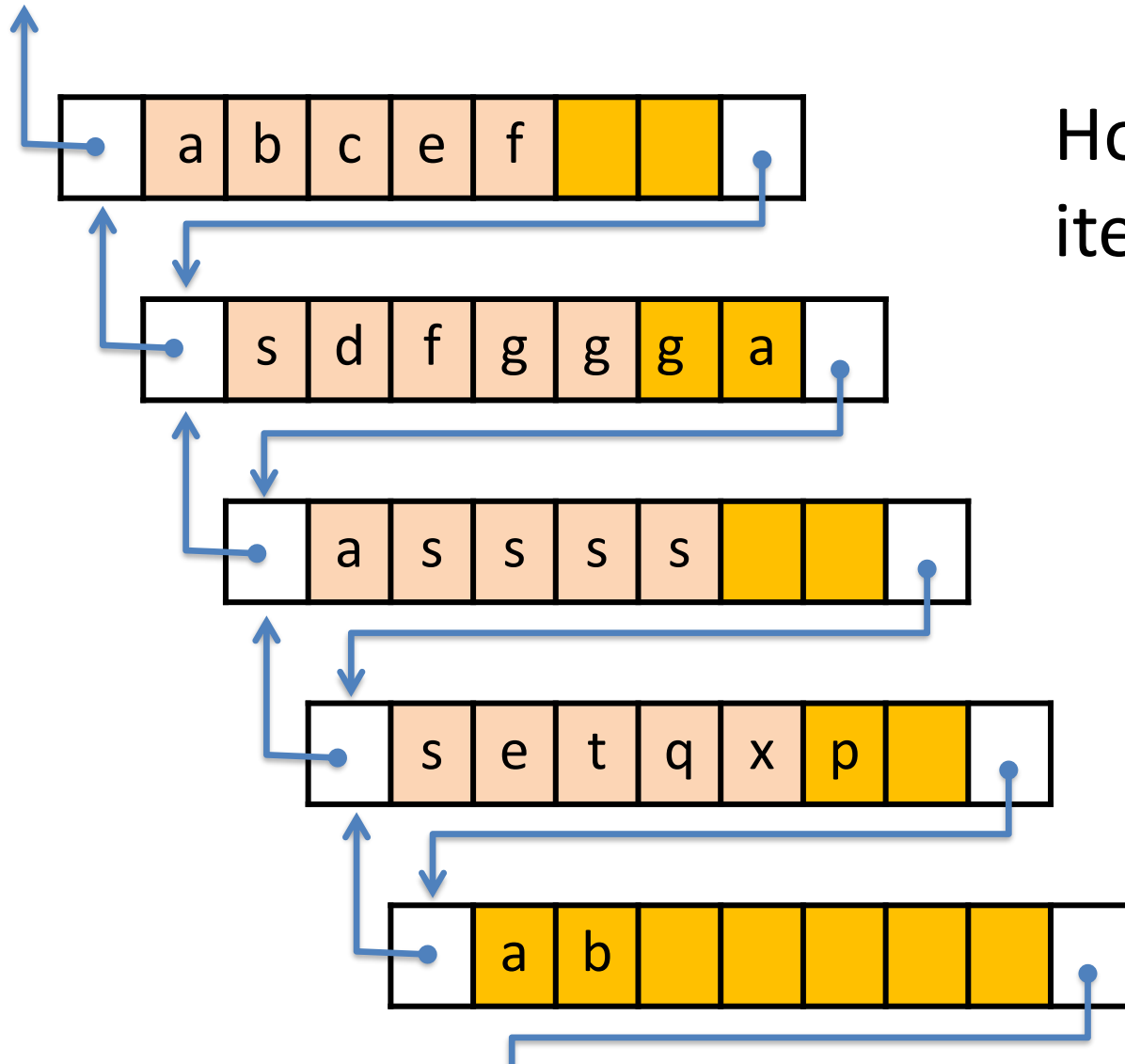
at most

$$\frac{n}{b-1} + 1 \in O(n/b)$$



Space-Efficient Linked List

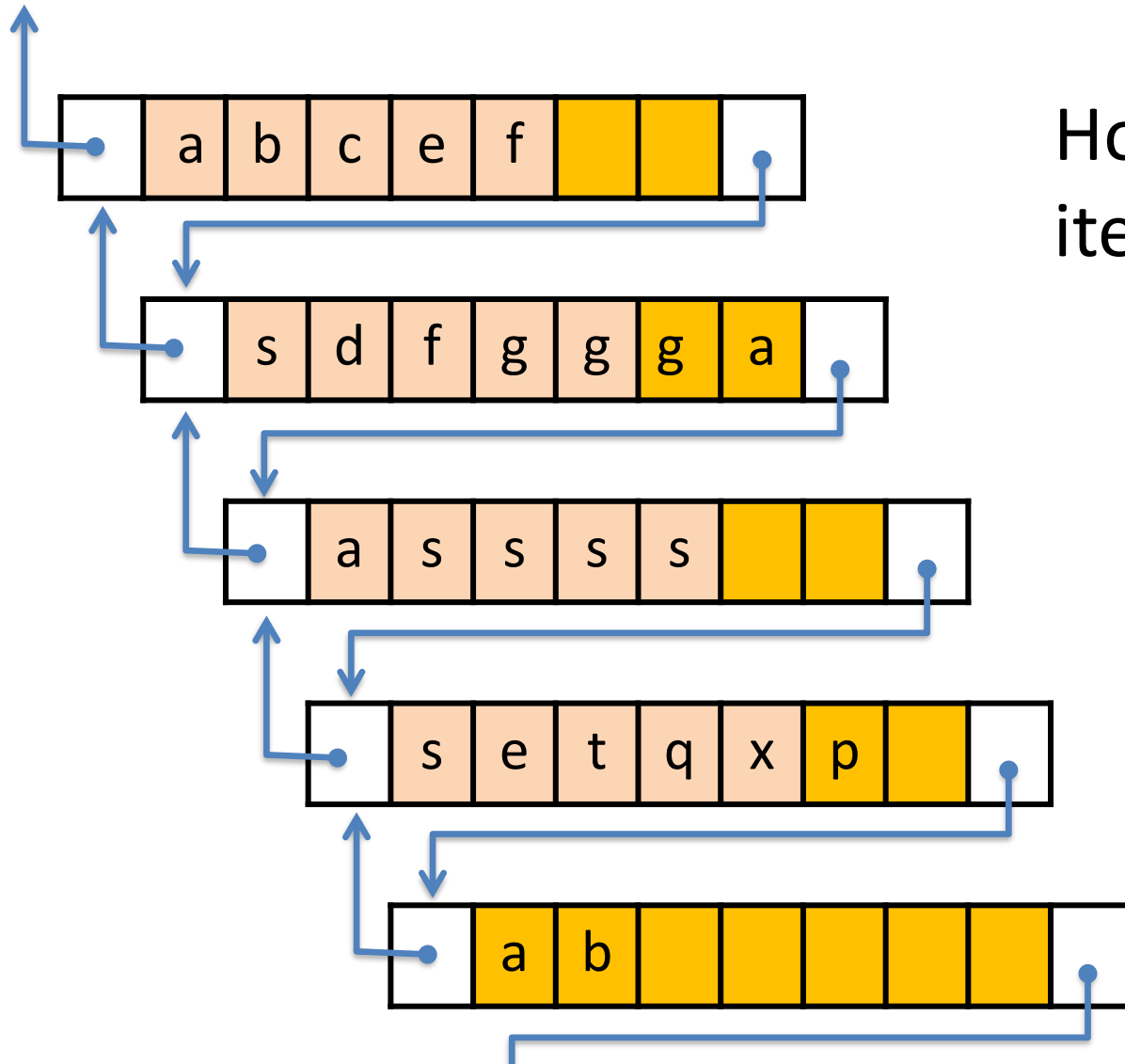
An **SEList** is a doubly-linked list of blocks



How do we find item *i* in the list?

Space-Efficient Linked List

An **SEList** is a doubly-linked list of blocks



How do we find item *i* in the list?

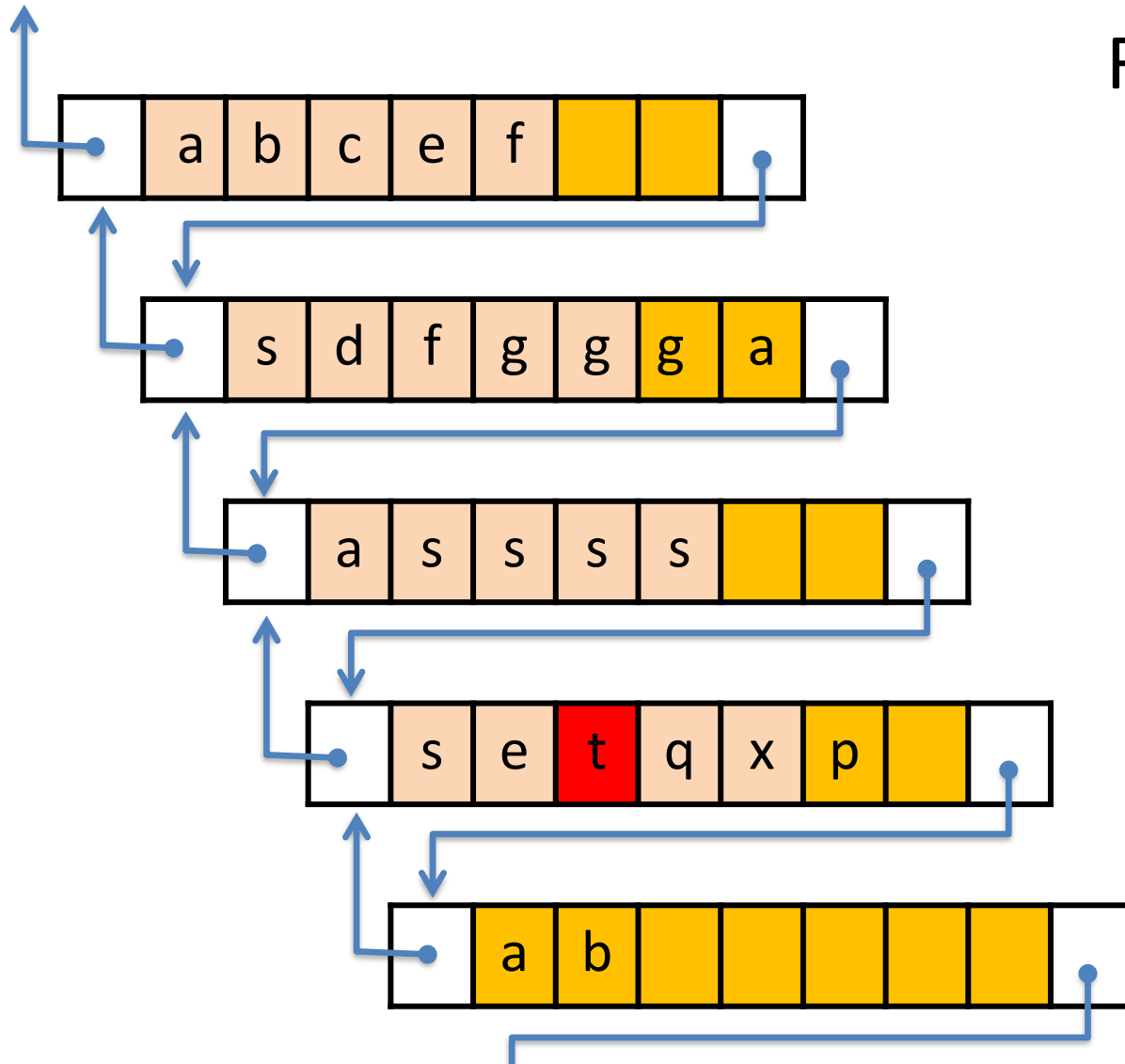
Location

Node <i>u</i>	
int	<i>j</i>

Space-Efficient Linked List

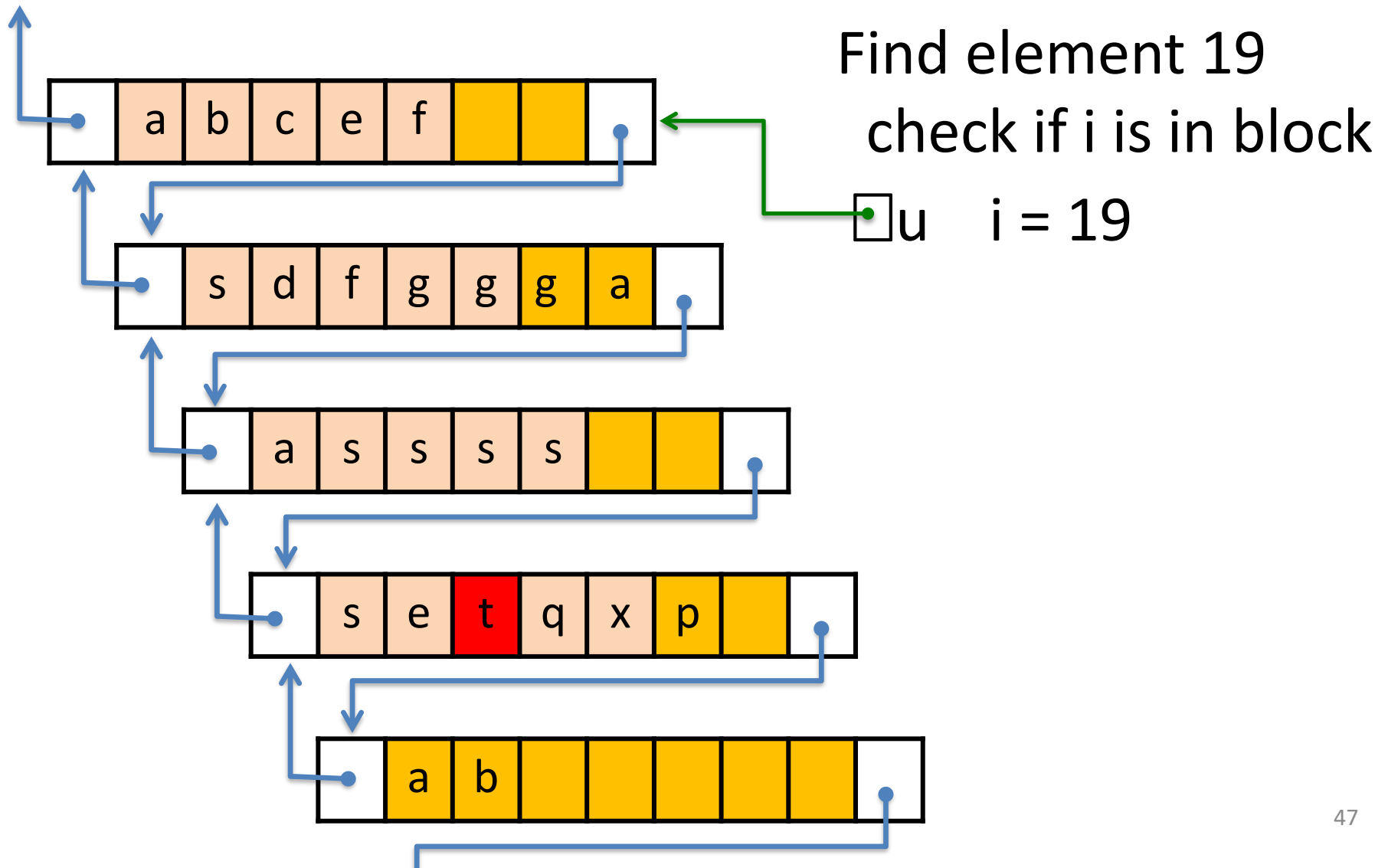
An **SEList** is a doubly-linked list of blocks

Find element 19



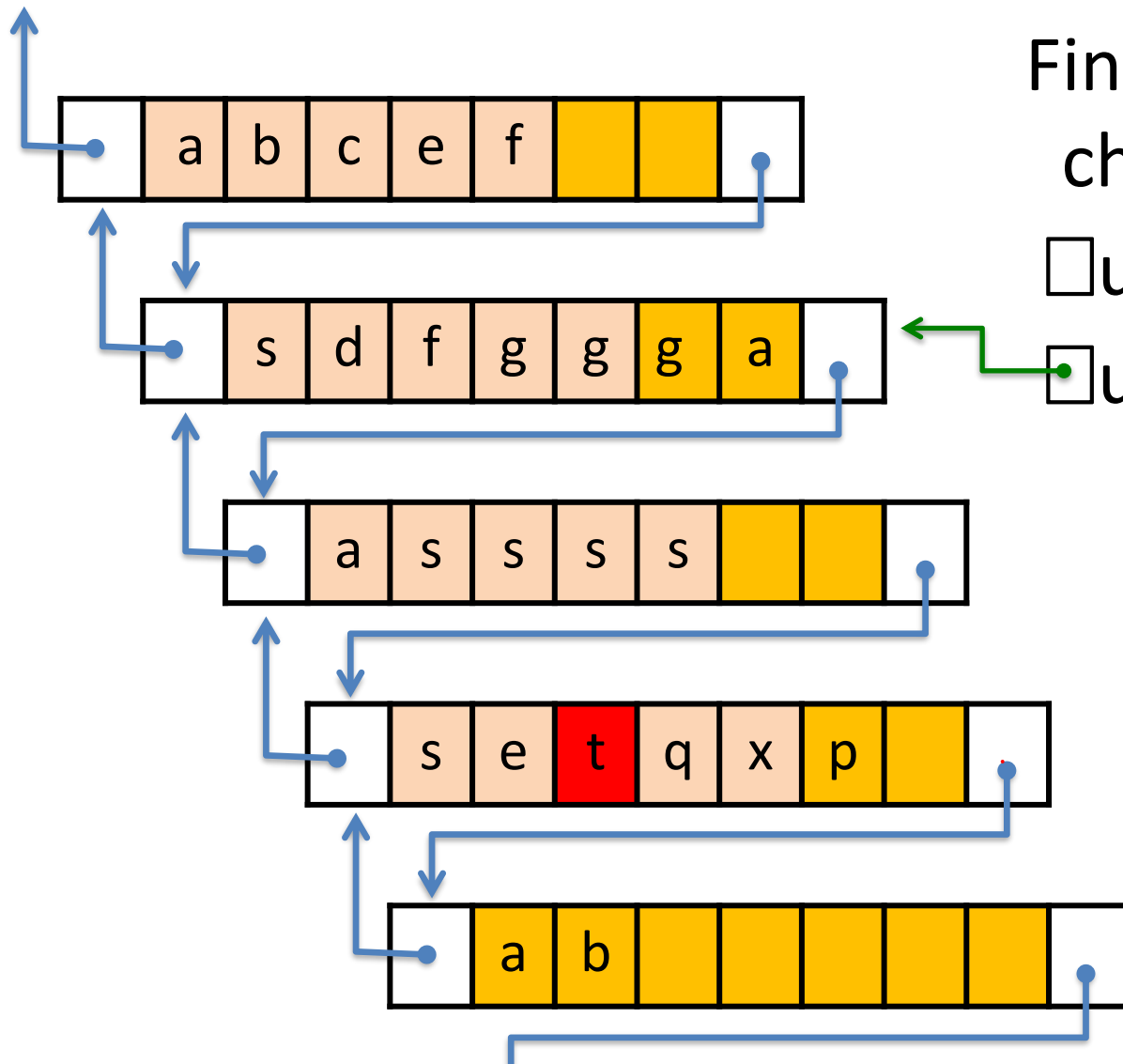
Space-Efficient Linked List

An **SEList** is a doubly-linked list of blocks



Space-Efficient Linked List

An **SEList** is a doubly-linked list of blocks



Find element 19

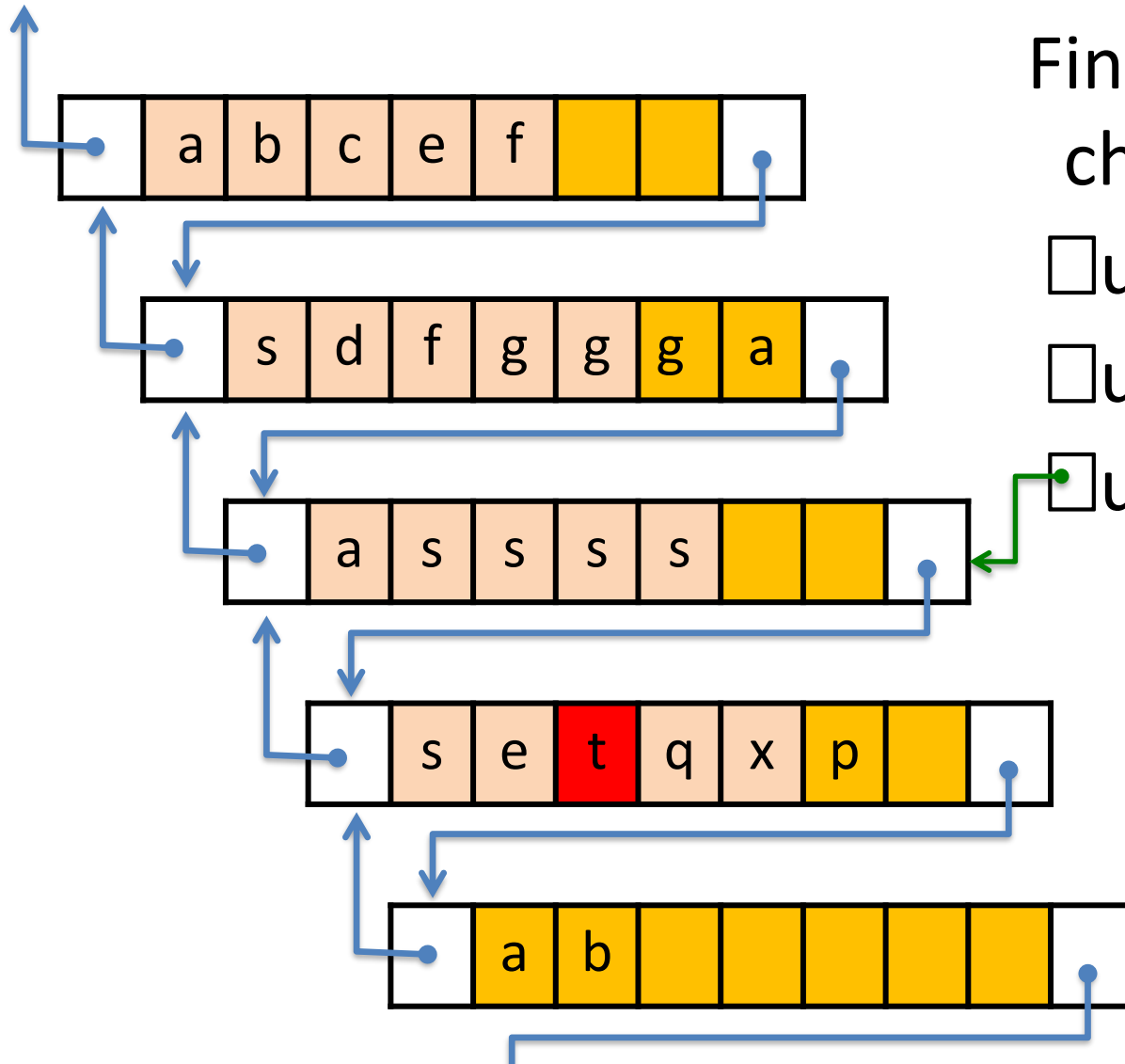
check if i is in block

☐ $u \quad i = 19$

☒ $u \quad i = 19 - 5 = 14$

Space-Efficient Linked List

An **SEList** is a doubly-linked list of blocks



Find element 19

check if i is in block

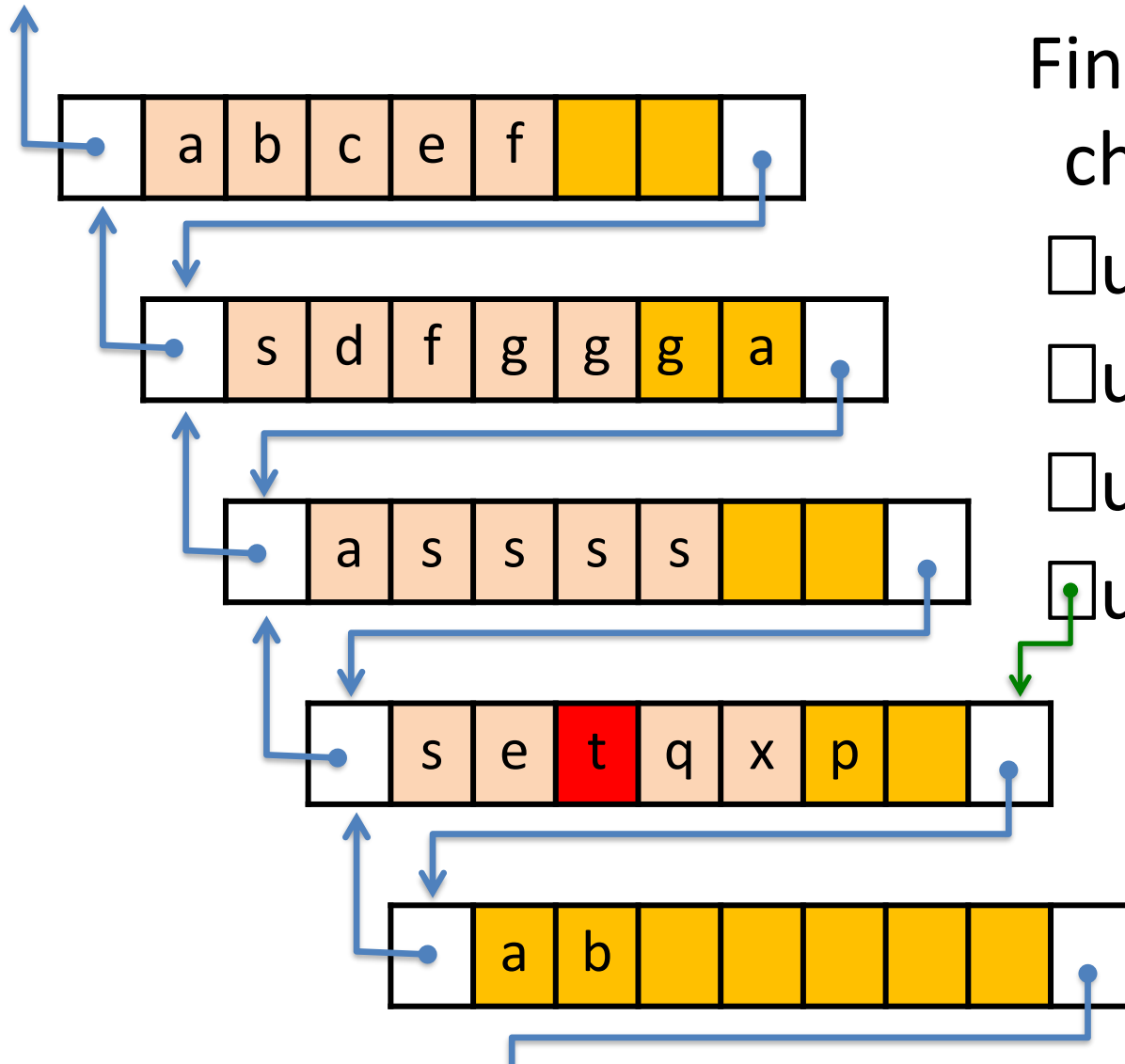
$$\square u \quad i = 19$$

$$\square u \quad i = 19 - 5 = 14$$

$$\square u \quad i = 14 - 7 = 7$$

Space-Efficient Linked List

An **SEList** is a doubly-linked list of blocks



Find element 19

check if i is in block

$$\square u \quad i = 19$$

$$\square u \quad i = 19 - 5 = 14$$

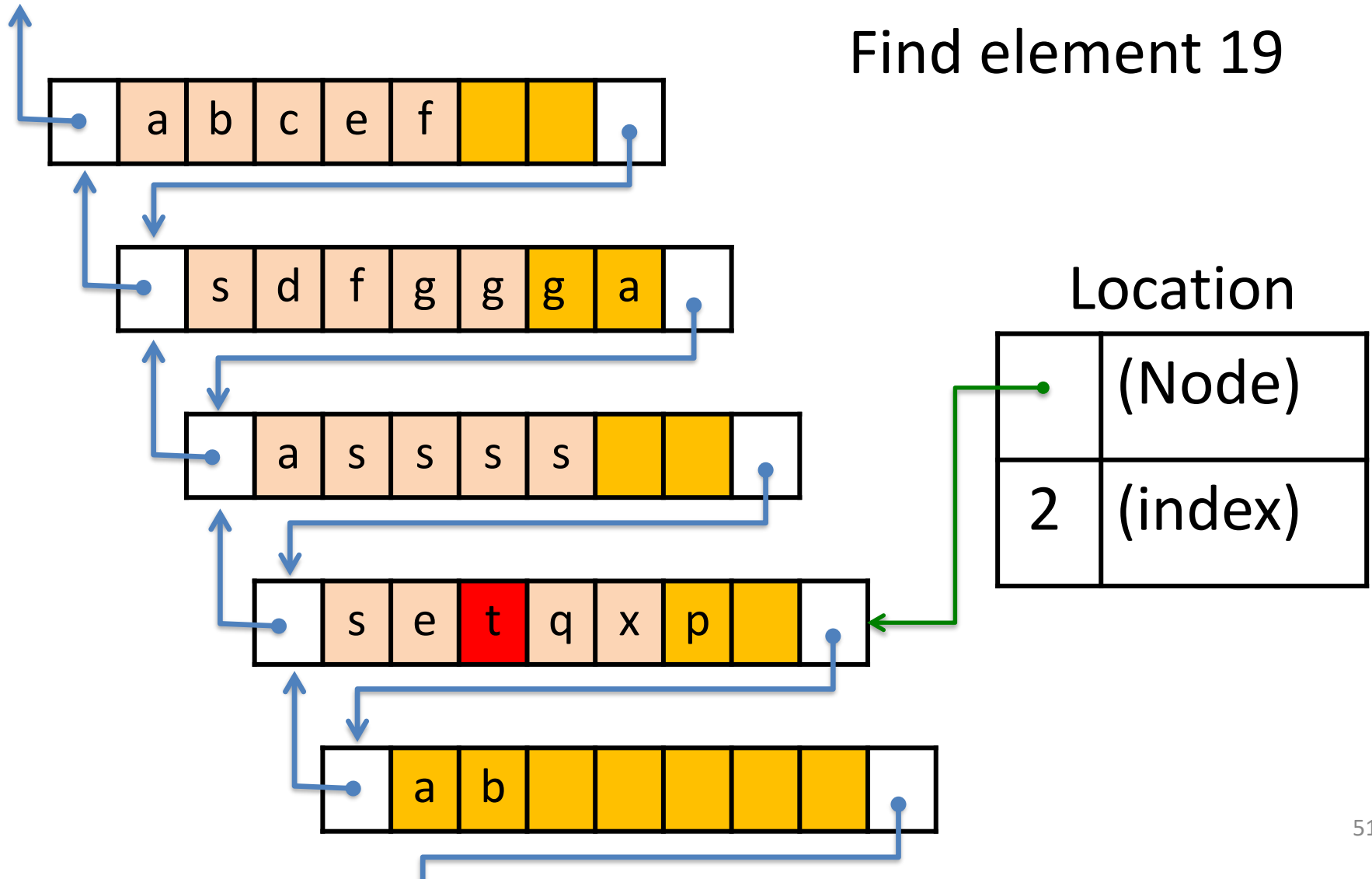
$$\square u \quad i = 14 - 7 = 7$$

$$\square u \quad i = 7 - 5 = 2$$

Space-Efficient Linked List

An **SEList** is a doubly-linked list of blocks

Find element 19



Space-Efficient Linked List

getLocation(i) returns a Location object that specifies where list item x_i is located

If $i < n/2$

- Start at the front and walk forwards

If $i \geq n/2$

- Start at the back and walk backwards

Space-Efficient Linked List

getLocation(i) returns a Location object that specifies where list item i is located in time

$$O(1 + \min\left\{\frac{i}{b}, \frac{n-i}{b}\right\})$$

Adding to an Selist

Use `getLocation(i)` to find block that i is in.

Since each block (except the last) must have $b - 1, b$ or $b + 1$ elements in it we need to consider several cases when adding a new element to the list

Let u_o be the block that i is in (found from `getLocation`)

Let $u_k = u_{k-1}.next$ for $k = 1, \dots, b - 1$

Never look past b blocks

Adding to an Selist

Case 1:

Case 1: we quickly ($r + 1 \leq b$) find a block that has room to add to it (size is $b-1$ or b) without reaching the end of the Selist

$u_0, \dots, u_{r-1}$ are all full ($b+1$ elements)

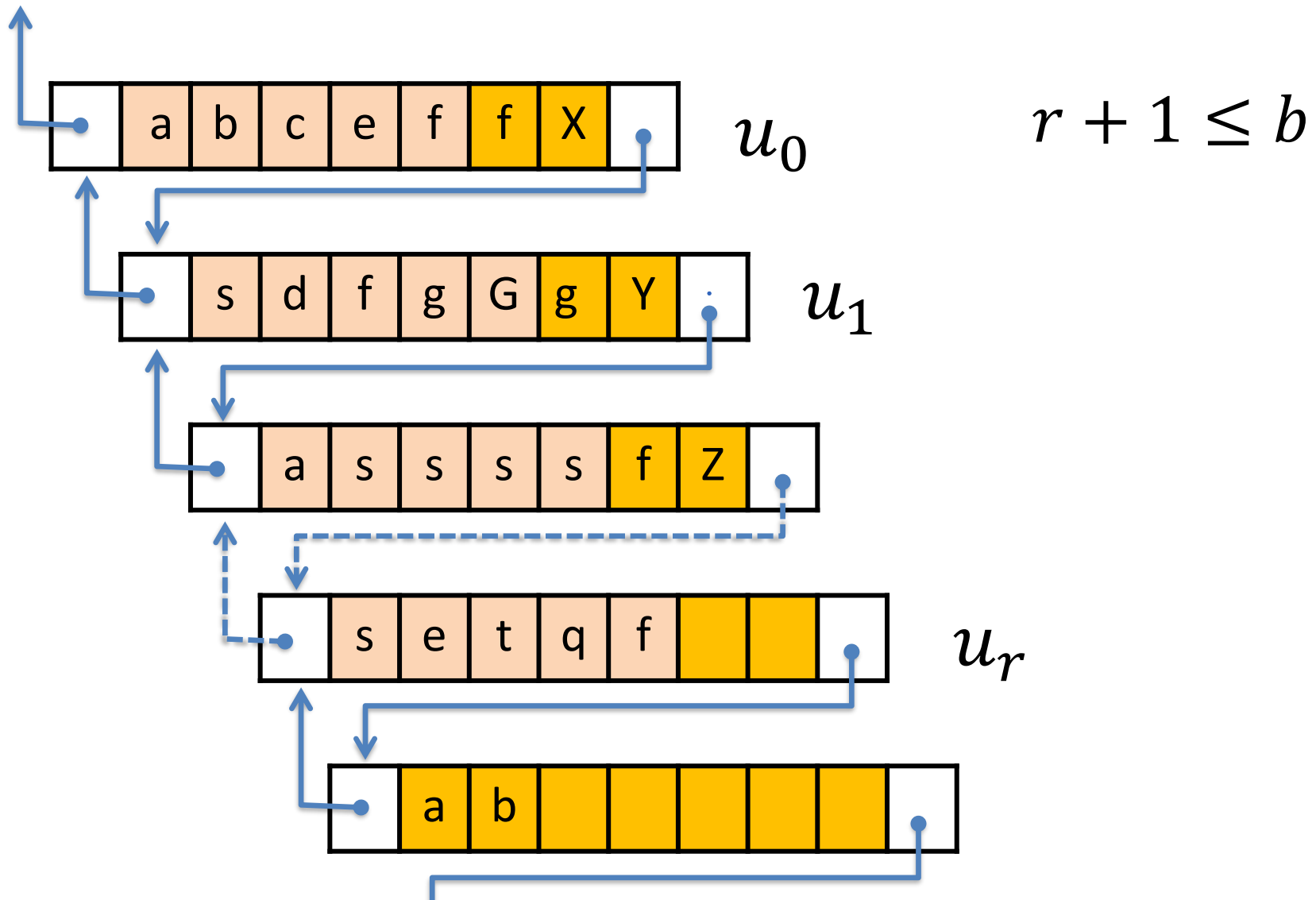
u_r is not full ($b-1$ or b elements)

Shift elements from block to block to free up space in u_0



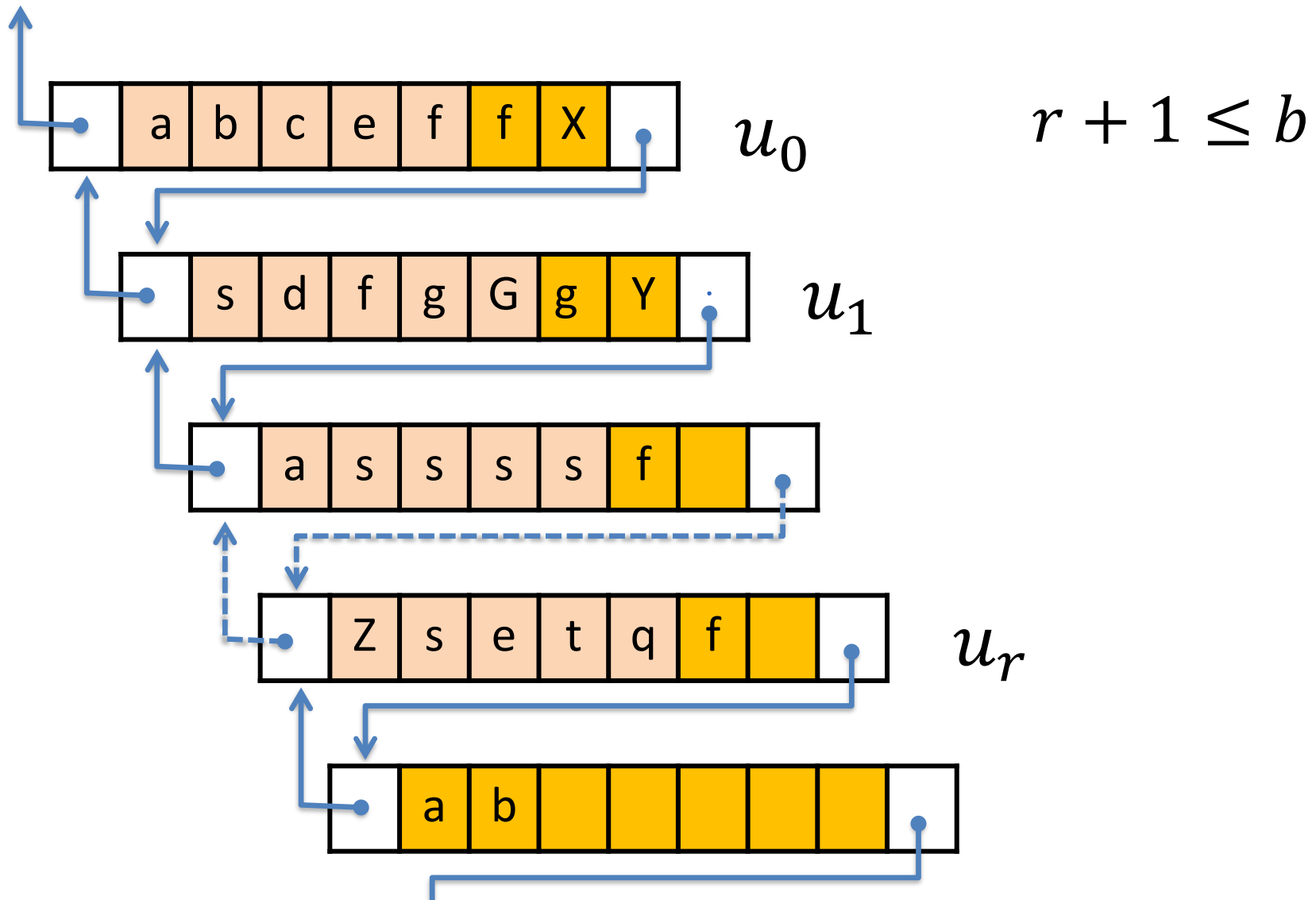
Adding to an Selist

Case 1:



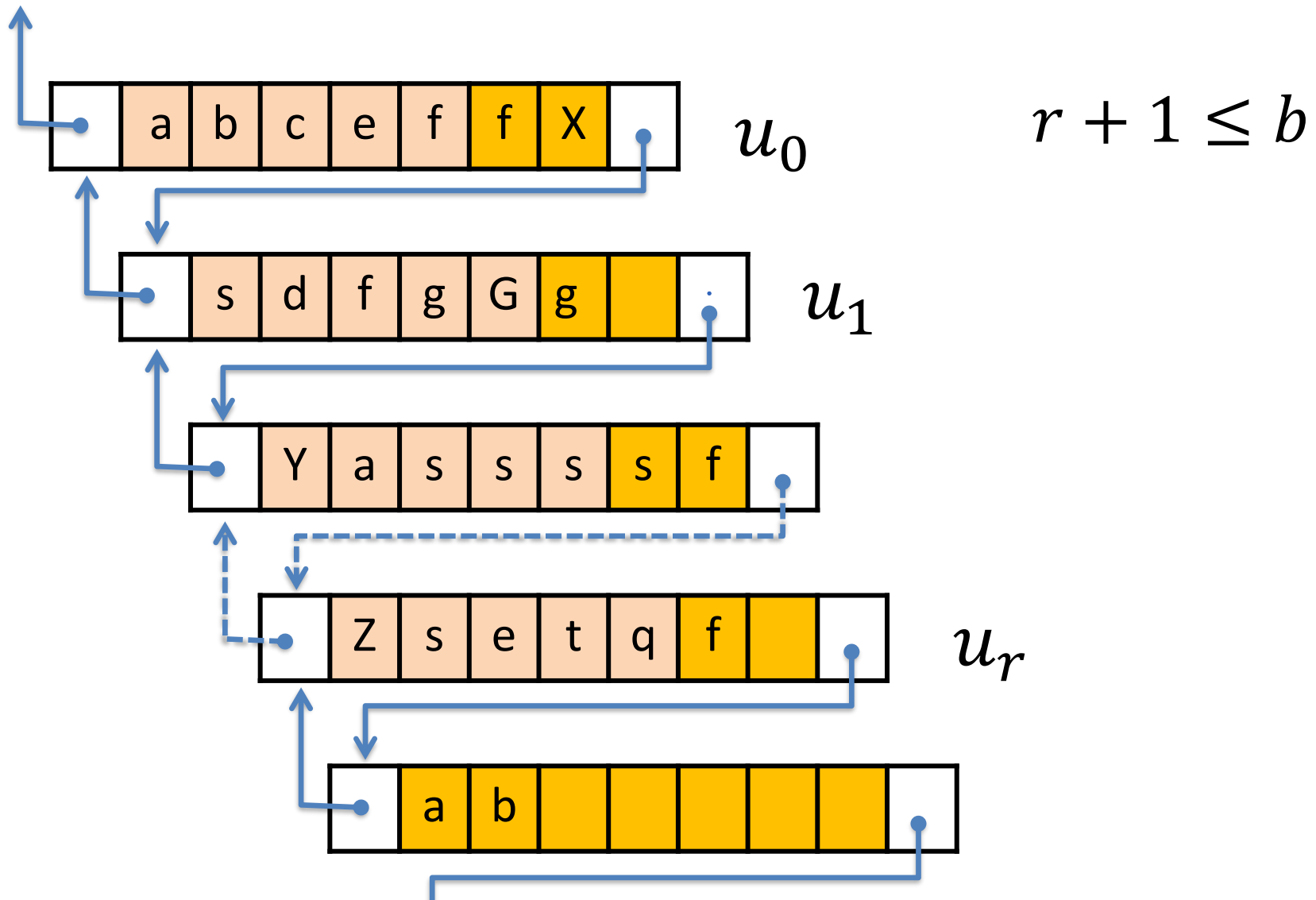
Adding to an Selist

Case 1:



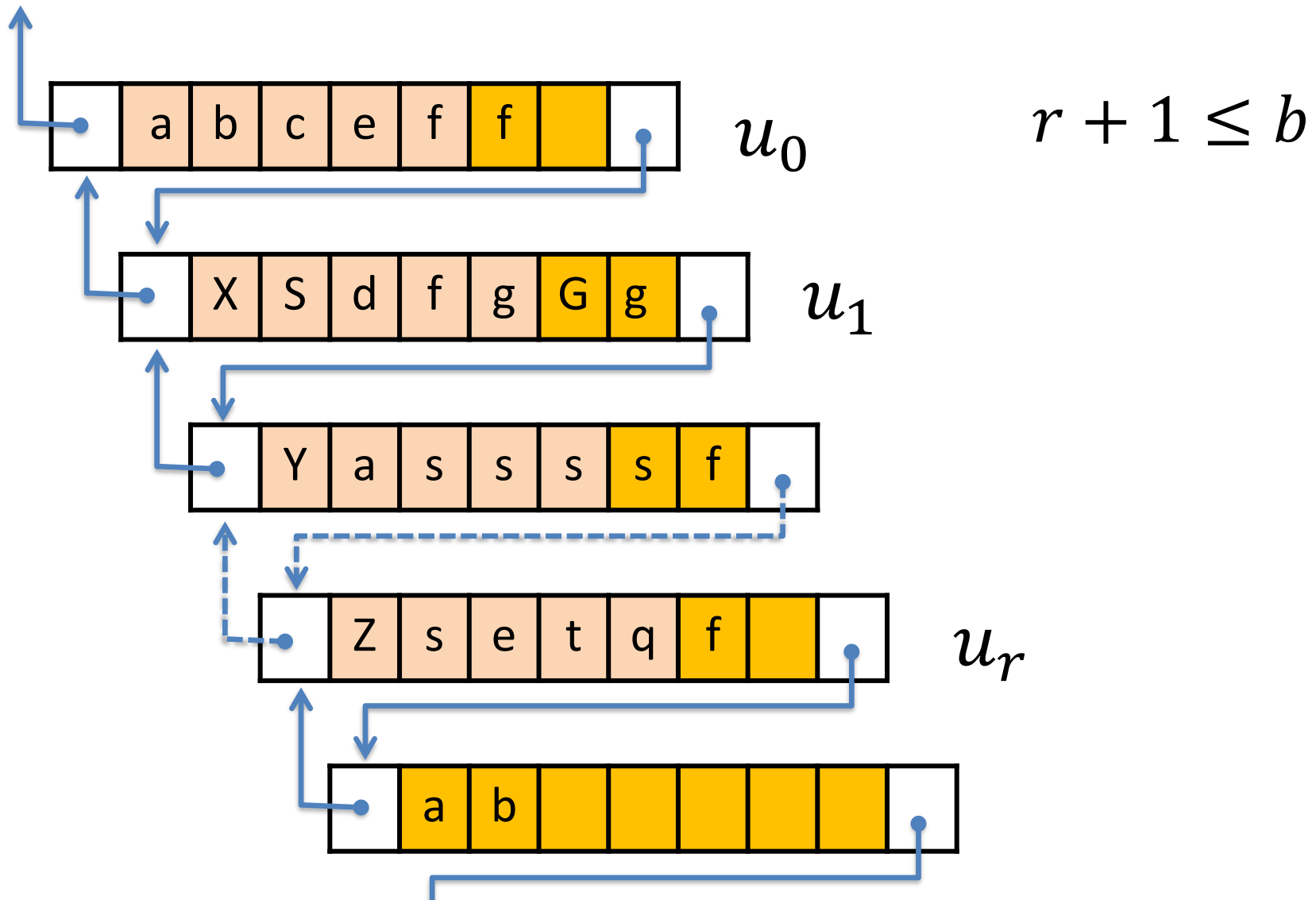
Adding to an Selist

Case 1:



Adding to an Selist

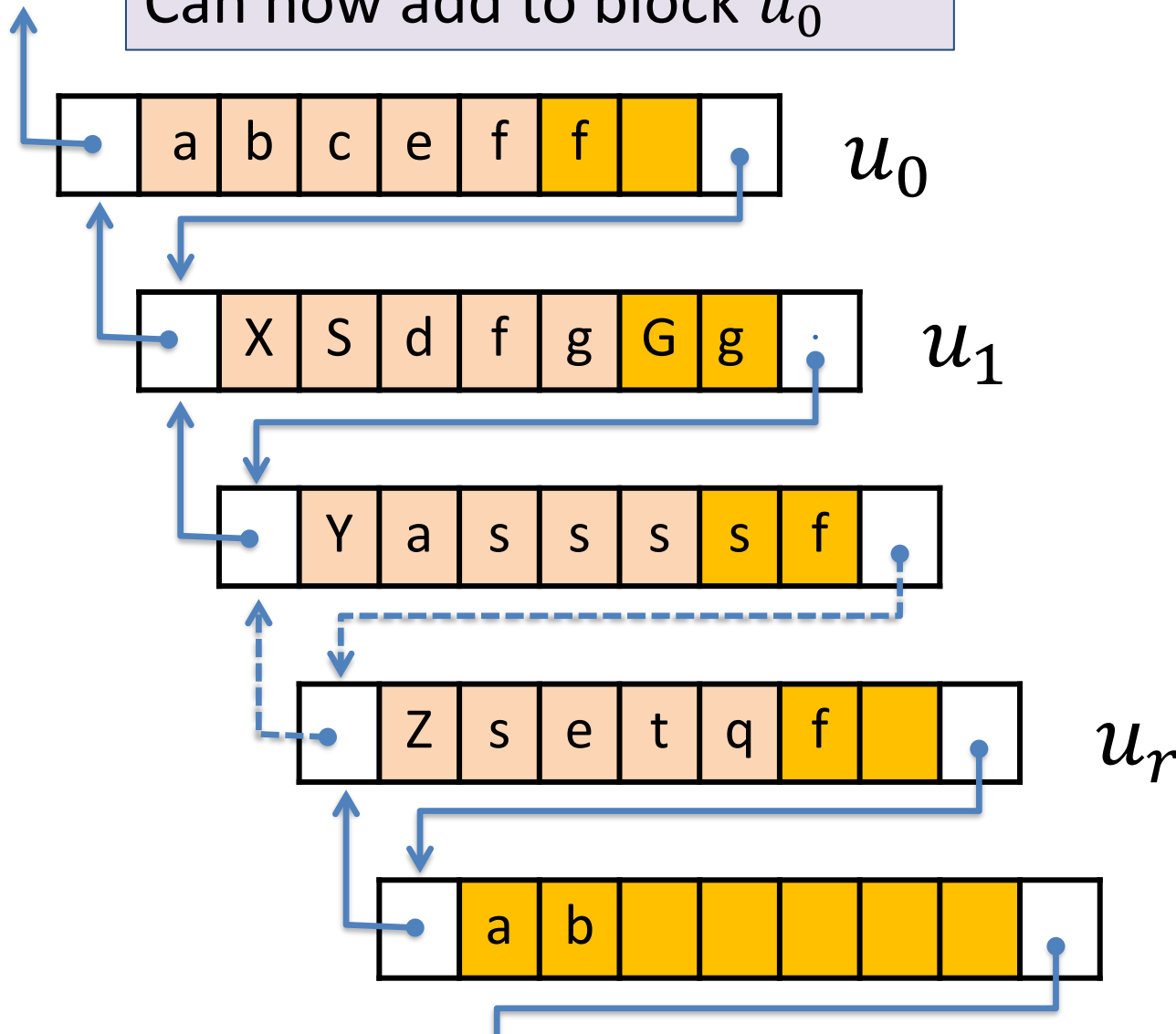
Case 1:



Adding to an Selist

Case 1:

Can now add to block u_0

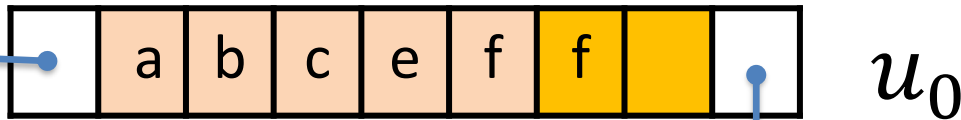


$$r + 1 \leq b$$

Adding to an Selist

Case 1:

Can now add to block u_0



$$r + 1 \leq b$$

What is cost?

r dequeues + r enqueues $\in O(r) \in O(b)$

insert into deque $\in O(b)$

Adding to an Selist

Case 2:



Case 2: we quickly ($r + 1 \leq b$) reach the end of the Selist before finding a block with room to add an element.

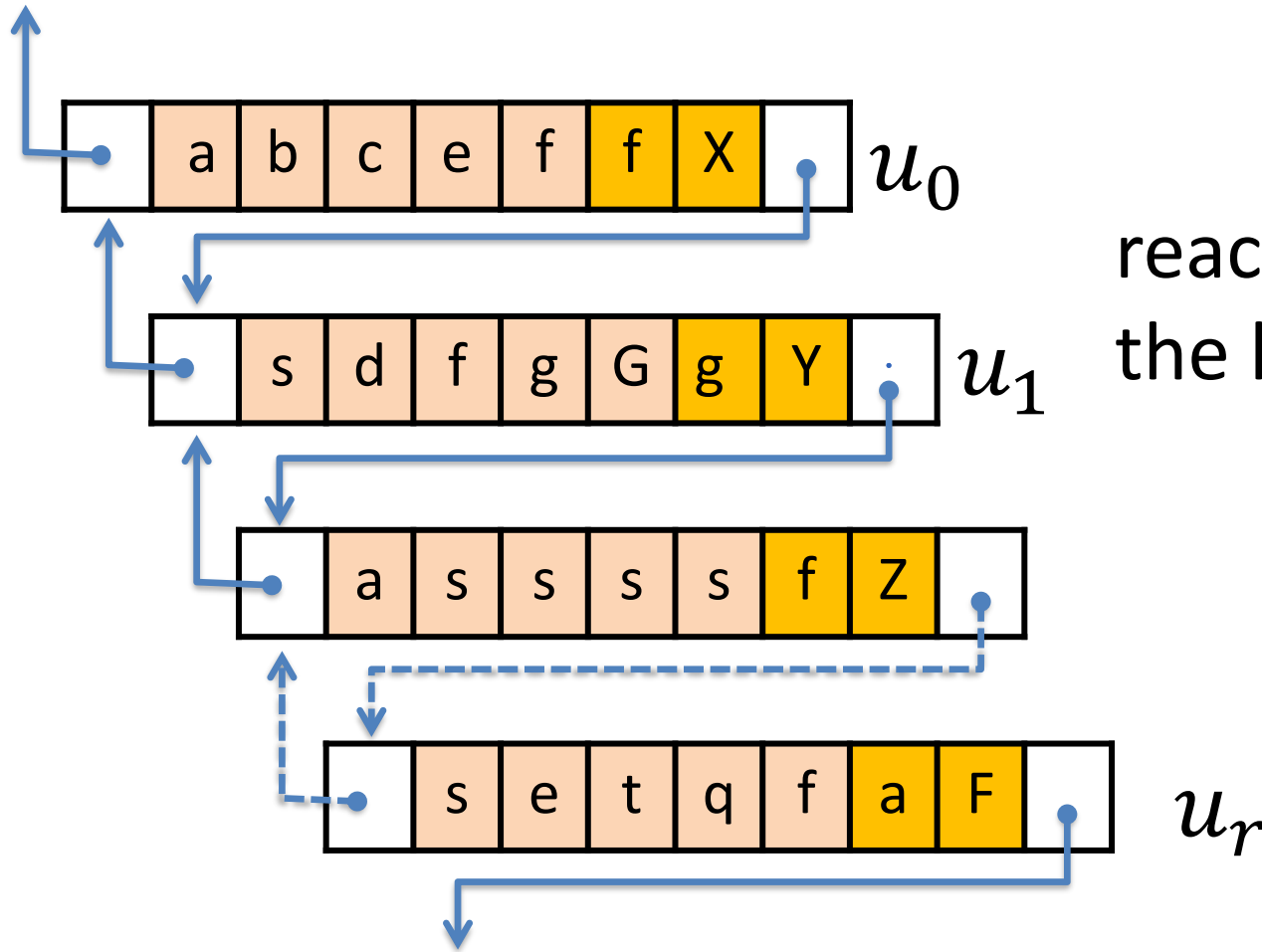
$u_0, \dots, u_r$ are all full ($b+1$ elements)

u_r is the last block

Add a new block and shift elements from block to block to free up space in u_0

Adding to an Selist

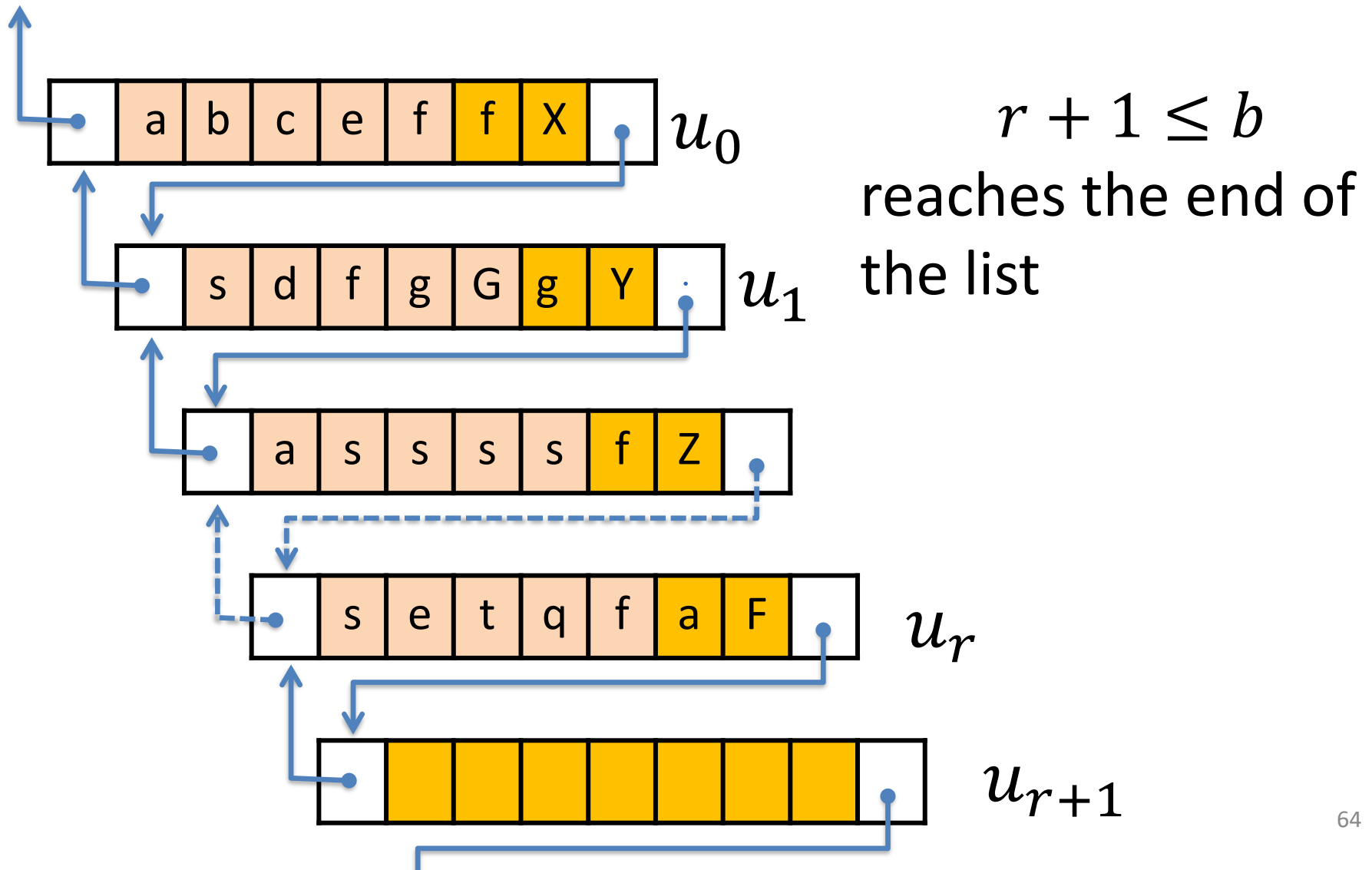
Case 2:



$r + 1 \leq b$
reaches the end of
the list

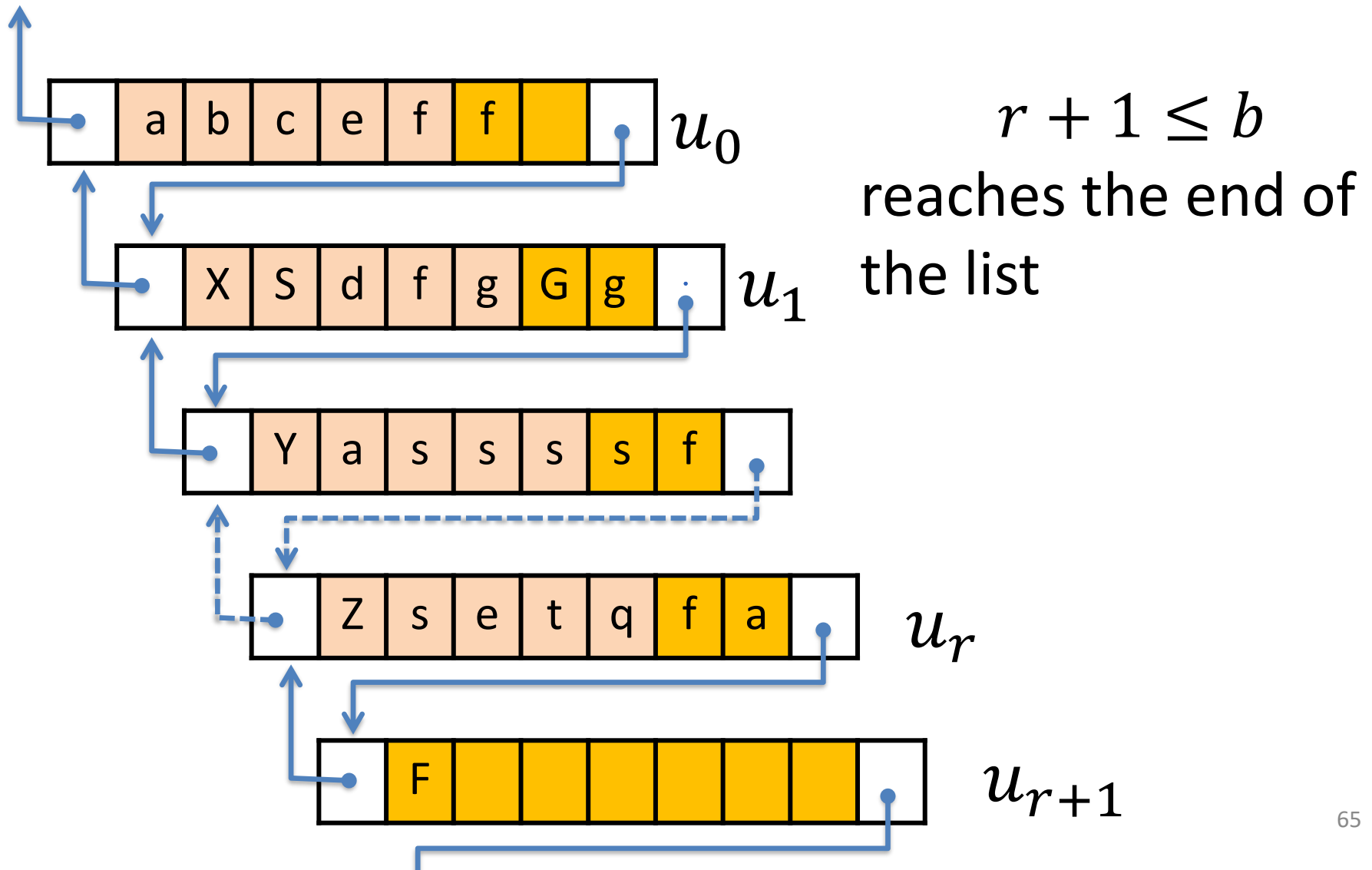
Adding to an Selist

Case 2:



Adding to an Selist

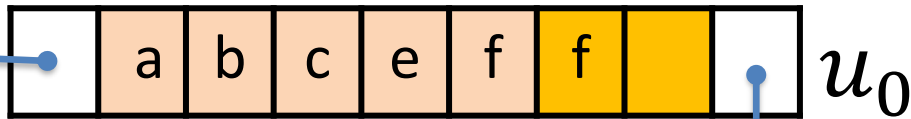
Case 2:



Adding to an Selist

Case 2:

Can now add to block u_0



$$r + 1 \leq b$$

reaches the end of

What is cost?

$r+1$ dequeues + $r+1$ enqueues $\in O(r) \in O(b)$

insert into deque $\in O(b)$



Adding to an Selist

Case 3:

Case 3: we do NOT quickly ($r + 1 \leq b$) find a block that has room to add to it (size of $b-1$ or b)

$u_0, \dots, u_r$ are all full ($b+1$ elements)



Adding to an Selist

Case 3:



Case 3: we do NOT quickly ($r + 1 \leq b$) find a block that has room to add to it (size of $b-1$ or b)

We have b blocks $u_0, \dots, u_r$ with exactly $b+1$ elements in each.

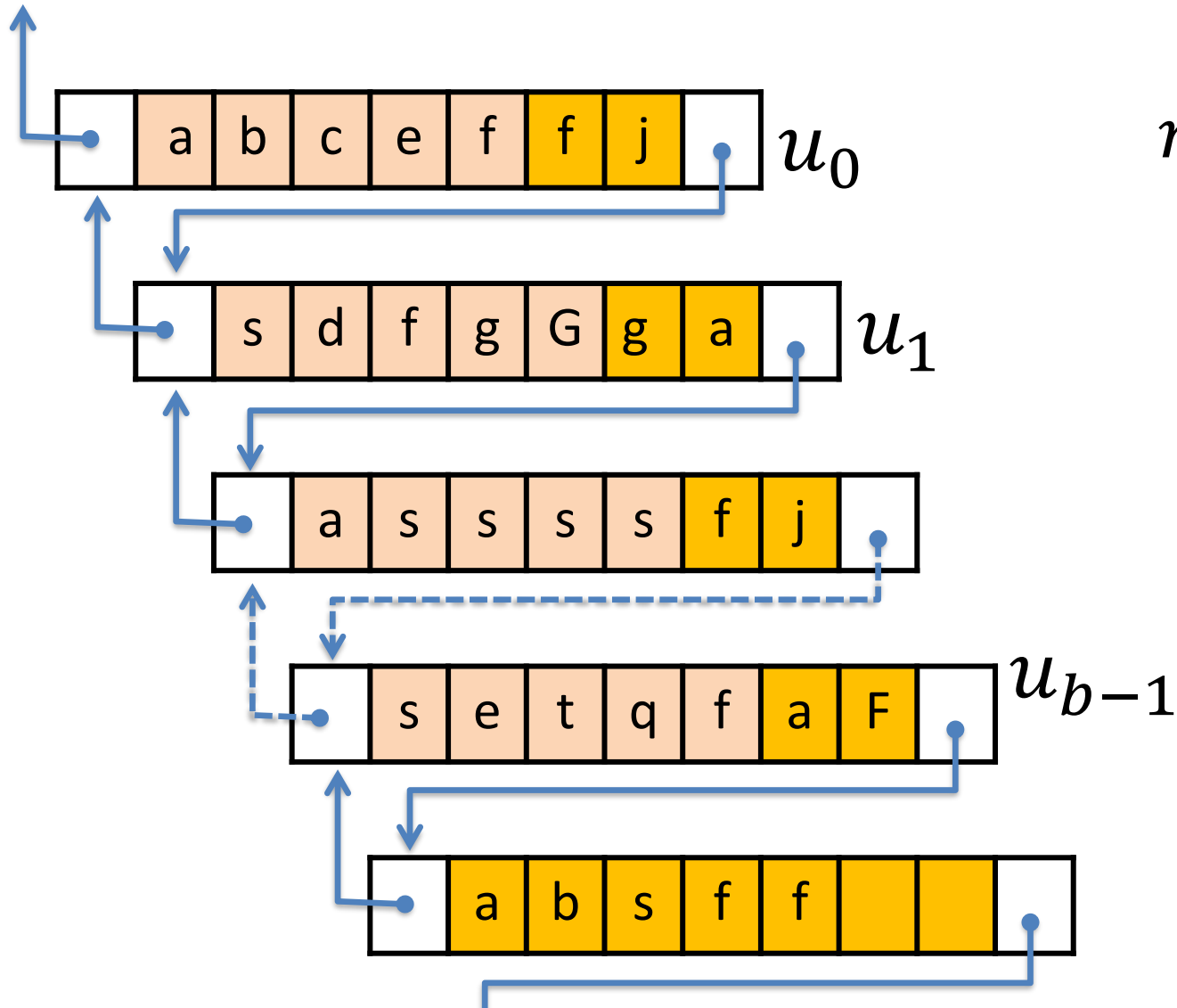
Insert a new empty block after u_r

Take these $b(b+1)$ elements and **spread** them into the $b+1$ blocks so that each has b elements

Adding to an Selist

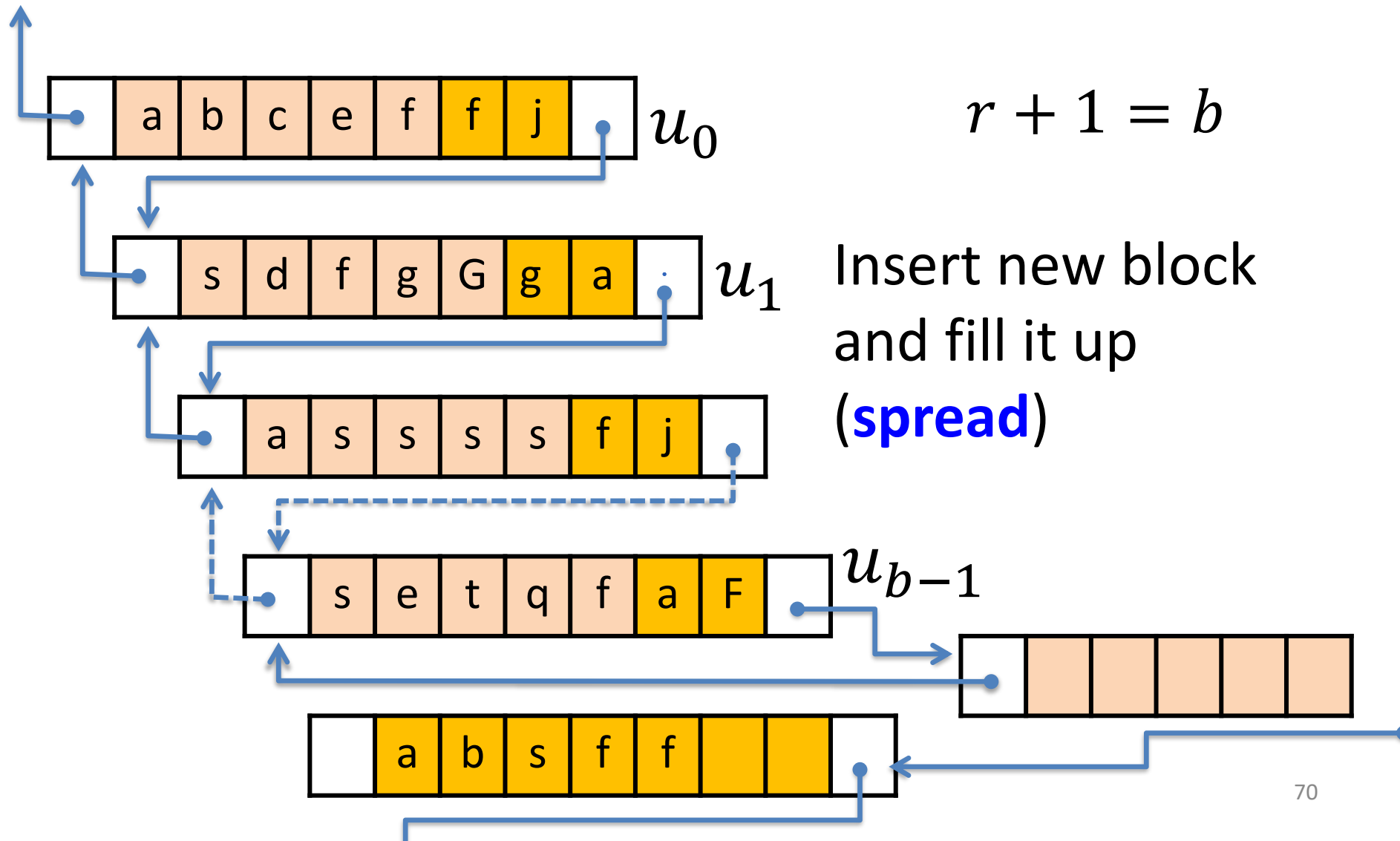
Case 3:

$$r + 1 = b$$



Adding to an Selist

Case 3:



Adding to an Selist

Case 3:

What is the cost?

Shift b elements from u_{b-1} to u_b

Shift $b-1$ elements from u_{b-2} to u_{b-1}

...

Shift 2 elements from u_1 to u_2

Shift 1 element from u_0 to u_1

Total $O(b^2)$ shifts



Adding to an Selist

add(i,x) costs

$$O(b + \min\{i, n - i\}/b)$$

If we ignore the spreading of case 3.

Removing from an Selist

remove(i) is similar to add (we need to consider cases)

Removing from an Selist

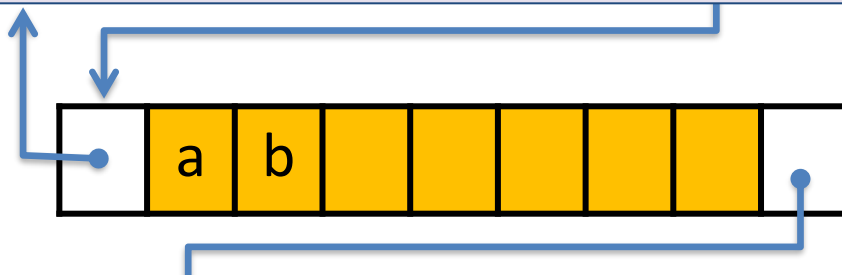
Case 1:

Case 1: we quickly ($r + 1 \leq b$) find a block that has b or $b+1$ elements

$u_0, \dots, u_{r-1}$ have $b-1$ elements

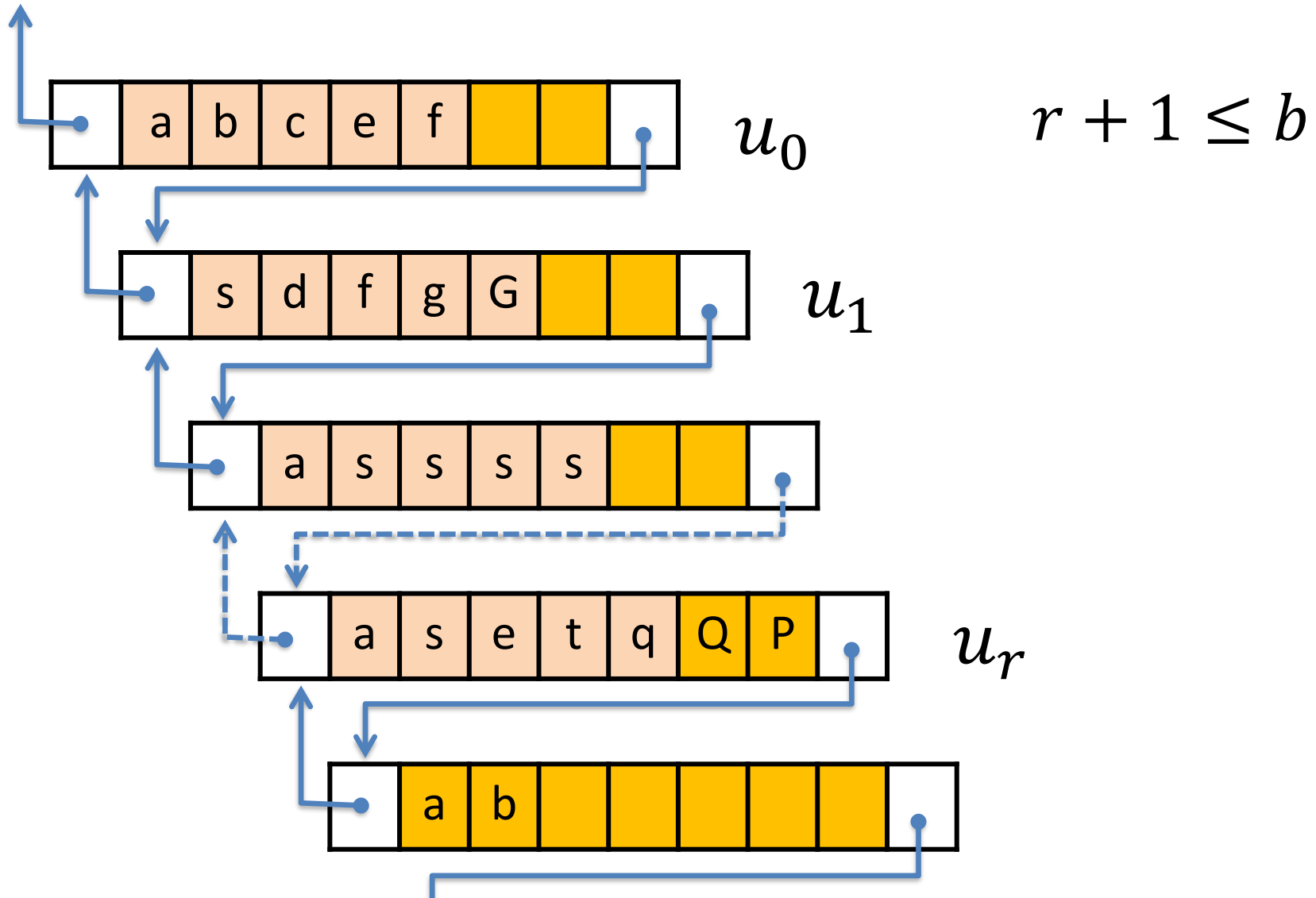
u_r has b or $b+1$ elements

Shift elements from block to block from u_r to u_0 so we can remove from u_0



Removing from an Selist

Case 1:



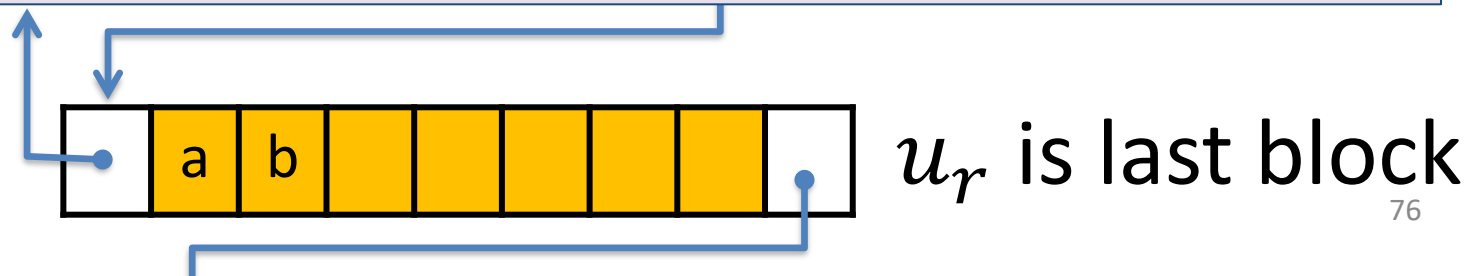
Removing from an Selist

Case 2:

Case 2: we quickly ($r + 1 \leq b$) reach the end of the Selist without finding a block with b or $b+1$ elements

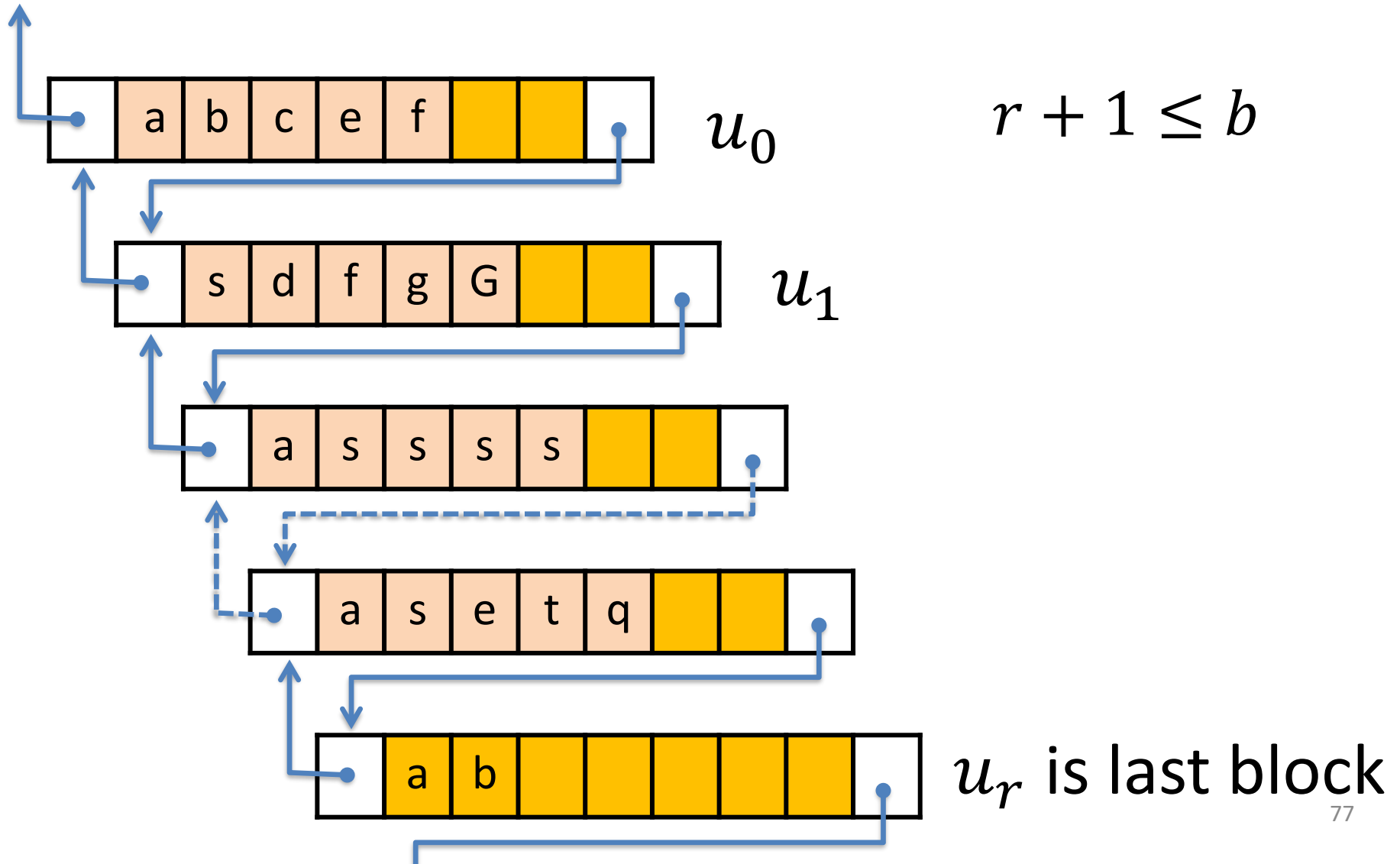
$u_0, \dots, u_r$ have $b-1$ elements

Shift elements from block to block from u_r to u_0 so we can remove from u_0



Removing from an Selist

Case 2:

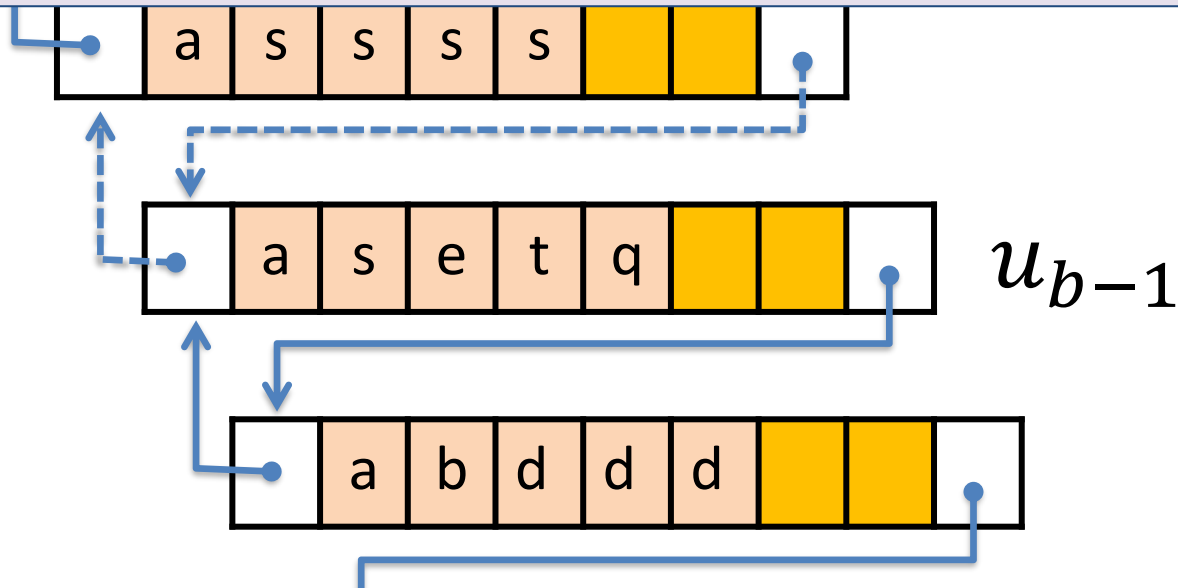


Removing from an Selist

Case 3:

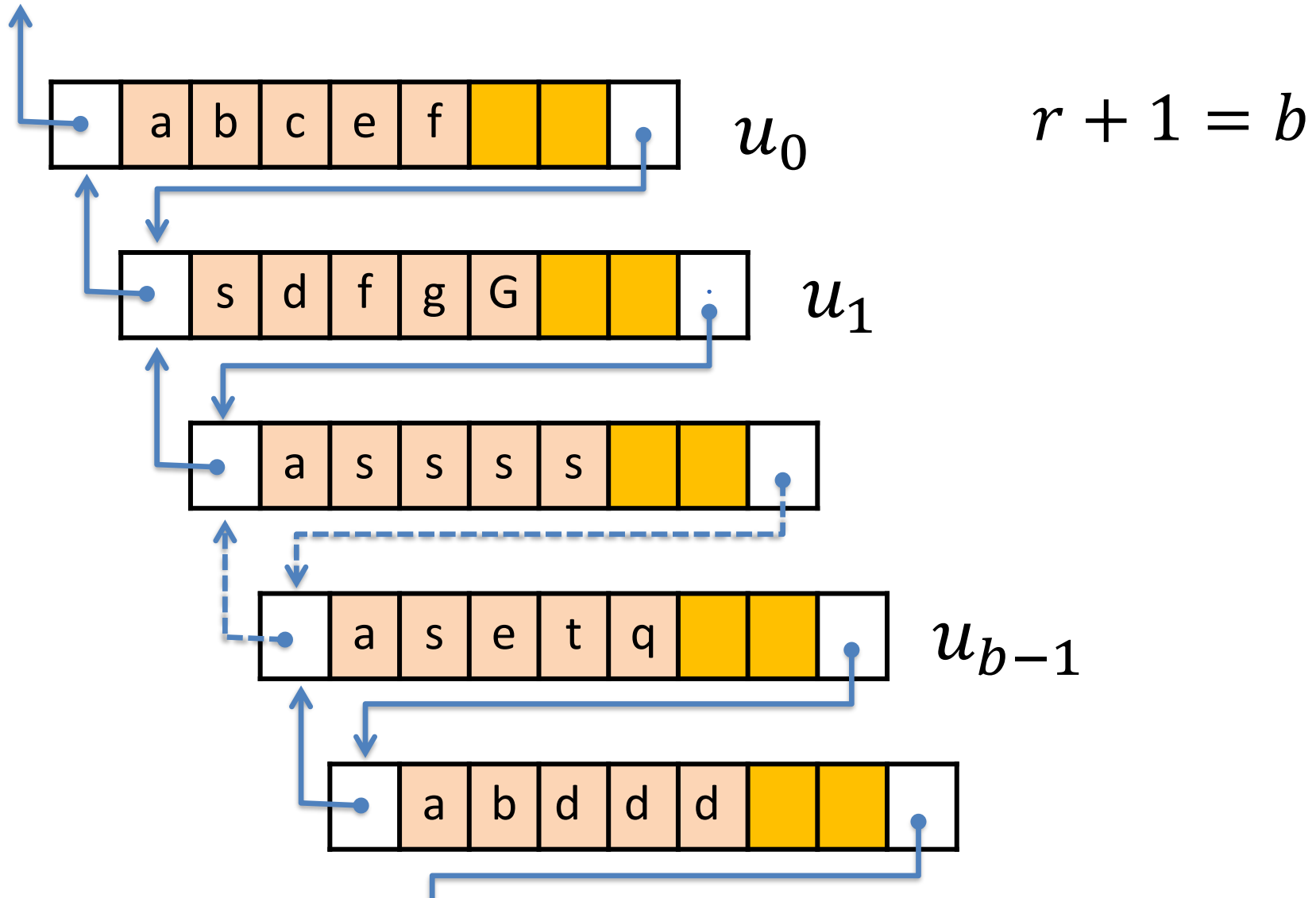
Case 3: we do NOT quickly ($r + 1 \leq b$) find a block that has b or $b+1$ elements

$u_0, \dots, u_{b-1}$ all have $b-1$ elements



Removing from an Selist

Case 3:



Removing from an Selist

Case 3:

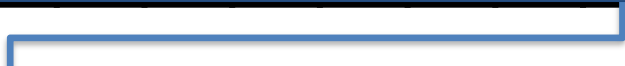


Case 3: we do NOT quickly ($r + 1 \leq b$) find a block that has b or $b+1$ elements

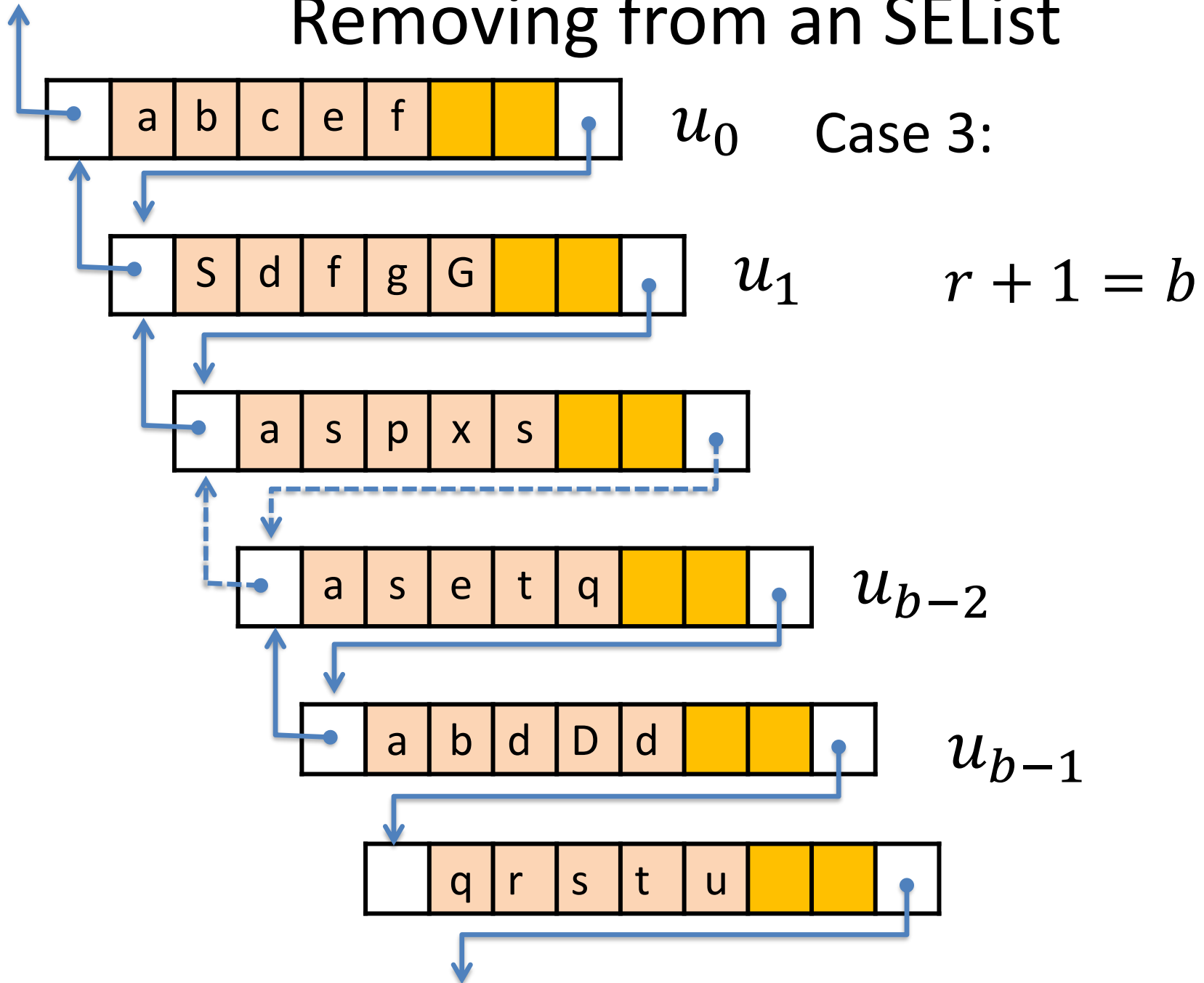
The b blocks $u_0, \dots, u_{b-1}$ each have $b-1$ elements ($b(b-1)$ elements in total)

gather these $b(b-1)$ elements into the $b-1$ blocks $u_0, \dots, u_{b-2}$ with b elements in each

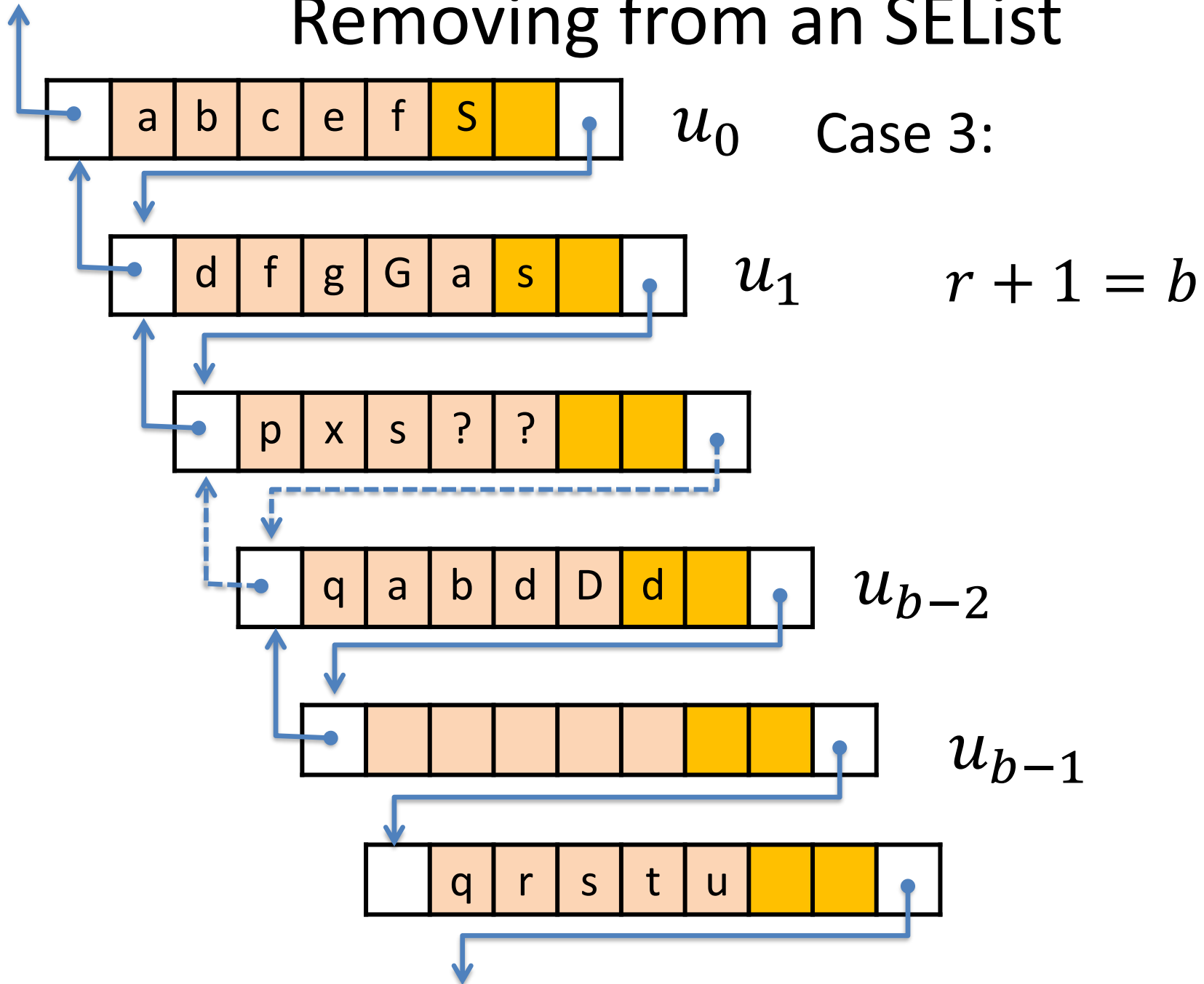
Delete u_{b-1}



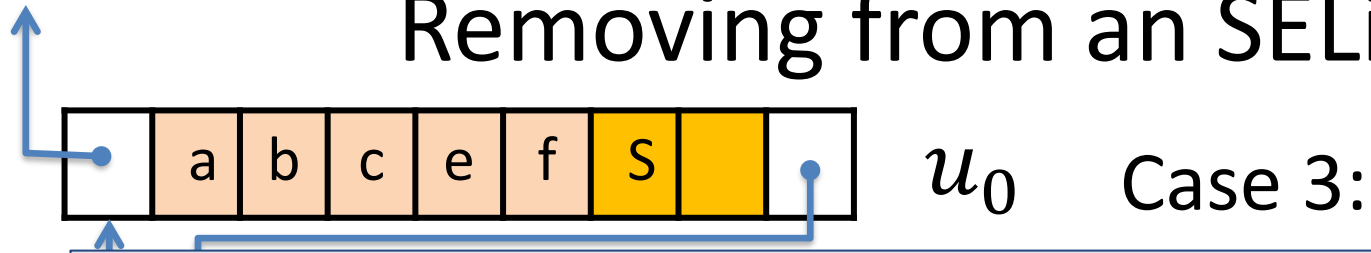
Removing from an Selist



Removing from an Selist



Removing from an SEList



How much does it cost to **gather**?

Add 1 element to u_0 from u_1

Add 2 elements to u_1 from u_2

Add 3 elements to u_2 from u_3

...

Add $b-1$ elements to u_{b-2} from u_{b-1}

Total of $b(b-1)$ elements to move before we can remove from u_0

Removing from an Selist

The cost of $\text{remove}(i)$ is

$$O(b + \min\{i, n - 1\} / b)$$

If we ignore the gathering of case 3

Cost of Spread/Gather

The spread and gather operations are each $O(b^2)$

But we don't call them very often. We'll show that there are $O(b)$ calls to add/remove between each call to gather or spread.

Cost of Spread/Gather

Let a block be called **fragile** if it does not contain exactly b elements

Let a block be called **rugged** if it contains exactly b elements

Define a potential

$$\Phi(s) = \#(\textit{fragile blocks})$$

Cost of Spread/Gather

Consider adding and spreading the data only

Each time you add an element,

- Case 1 : at most 1 rugged block becomes fragile
- Case 2 : new fragile block is created
- Case 3 : $b-1$ fragile blocks become rugged

Since the potential must always be non-negative, it means that there had to be at least $b - 1$ case 1 or case 2 adds before a case 3 add.

Cost of Spread/Gather

Consider adding and spreading the data only

Each time you add an element,

- Case 1 : $\Phi \leftarrow \Phi + 1$ at most
- Case 2 : $\Phi \leftarrow \Phi + 1$
- Case 3 : $\Phi \leftarrow \Phi - (b - 1)$

Since the potential must always be non-negative, it means that there had to be at least $b - 1$ case 1 or case 2 adds before a case 3 add.